



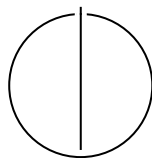
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Exploring and Improving Fault Localization in LLM Inference Engines

Tianhao Gu





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

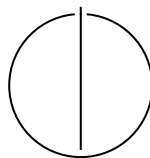
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Exploring and Improving Fault Localization
in LLM Inference Engines**

**Untersuchung und Verbesserung der
Fehlerlokalisierung in
LLM-Inferenzsystemen**

Author: Tianhao Gu
Examiner: Prof. Dr. Chunyang Chen
Supervisor: Dr. Mark Huasong Meng
Submission Date: 17.08.2026



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 17.08.2026

Tianhao Gu

Acknowledgments

As I bring my Master's thesis to completion, I would like to express my deepest and most sincere gratitude to everyone who has offered me guidance, support, and encouragement throughout my academic journey and research.

First and foremost, I would like to extend my heartfelt gratitude to my examiner, **Prof. Dr. Chunyang Chen**, for his guidance in shaping the broader academic direction of this work, as well as for his continued interest in and support for this research.

I am also profoundly grateful to my supervisor, **Dr. Mark Huasong Meng**, and to **Dr. Hao Guan**. Throughout the research and writing process, I benefited immensely from their insightful guidance, regular discussions, and highly constructive advice. Whenever I encountered conceptual obstacles or technical challenges, their patient guidance and constant encouragement gave me the reassurance and confidence to devote myself wholeheartedly to this research.

Above all, I offer my deepest gratitude to my parents. No matter where life takes me, they remain the two people in this world who believe unconditionally that I am capable of achieving anything. Their unwavering trust, understanding, and support have been my strongest foundation and an enduring source of strength in the face of every challenge. You may not be able to read these words in English, but I know deep in my heart that it was the countless times you cycled through wind and rain to pick me up and bring me home that gave me the opportunity to stand where I am today and to see a wider world.

The world has no shortage of people who work hard, yet among them, I have been one of the fortunate ones. This thesis is not mine alone; we completed it together. Therefore, I would like to dedicate the thesis to everyone who has helped me along the way, including those whose names may not appear on these pages but whose kindness I will always remember.

Tianhao Gu
Munich, 08.2026

Abstract

Large language model (LLM) inference engines are repository-scale systems that coordinate model management, request serving, resource-aware scheduling, native execution, hardware adaptation, and build and deployment logic. An issue describes where a failure is observed, not necessarily where the responsible implementation resides, so a symptom may be repaired in a different type of artifact than the one in which it appears. Existing research characterizes the symptoms, root causes, and repair patterns of inference-engine bugs, or diagnoses reproducible failures at runtime, but does not rank the concrete files that developers modify in the corresponding pre-fix repository. To the best of our knowledge, no prior work has formulated and systematically evaluated this issue-guided, execution-free, file-level fault-localization task for bugs in LLM inference-engine software, so it remains unknown how effectively existing localization approaches can address this problem.

This thesis addresses that gap using Ollama, a widely adopted LLM inference engine, as a case study, and a dataset of 145 validated issue-fix pairs with unambiguous, patch-derived single-file repair labels. We evaluate two representative information-retrieval-based (IR-based) localizers (BugLocator, BLUiR) and an LLM-agent-based approach (SWE-agent) under two backbone models in a common execution-free setting, in which a localizer receives an issue report and its pre-fix repository snapshot but does not compile the repository, run tests or models, or collect new runtime traces. We find that the IR-based approaches give imprecise but useful candidate reduction, ranking fewer than one quarter of repair files first while about half fall within the Top-5, whereas both SWE-agent configurations produce substantially more top-heavy rankings, reaching up to 64.83% Top-1 (94 of 145 cases) and 88.97% Top-5 (129 of 145 cases). The two approach families expose a clear trade-off between ranking effectiveness and computational cost, and manual coding of LLM-agent failure trajectories attributes localization misses to three recurring, identifiable pitfalls: evidence non-reachability, causal-chain truncation, and non-source-artifact under-inspection, rather than to unexplainable model behavior.

Motivated by these bottlenecks, we then present CLUEFL (*Cross-Software-Layer Understanding of Evidence for Fault Localization*), a domain-specific fault-localization framework that combines project-specific software-layer knowledge, historical issue-fix retrieval, and a root-cause taxonomy during hypothesis generation, together with purpose-built tools for tracing cross-layer dependencies, retrieving code metadata, and inspecting indi-

vidual functions during evidence exploration, before a final file-level relevance-ranking stage. Under matched backbone conditions, CLUEFL improves early prioritization on both evaluated backbones: on GPT-5.2, it raises Top-1 by 7.59 percentage points (74 to 85 of 145 cases) and Top-5 by 6.21 percentage points (109 to 118 of 145 cases); on DeepSeek V4 Pro, it raises Top-1 by 2.76 percentage points (94 to 98 of 145 cases) while Top-5 coverage remains unchanged. The more limited gain for the DeepSeek V4 Pro configuration coincides with its higher observed SWE-agent baseline scores, which leave less room for improvement. The largest observed benefit occurs for the GPT-5.2 configuration, whose baseline scores are lower.

This thesis makes three contributions: (1) the first systematic evaluation of issue-guided, execution-free file localization for LLM inference-engine bugs, comparing representative IR-based and LLM-agent-based approaches in effectiveness, cost, backbone models, and failure behavior; (2) a repository-derived evaluation dataset of 145 validated Ollama issue-fix pairs with pre-fix snapshots and single-file repair labels, released as a reusable foundation for future research; and (3) CLUEFL, a domain-specific localization framework that demonstrates that external knowledge and a well-designed workflow can improve file-ranking effectiveness for LLM inference-engine bugs.

Contents

Acknowledgments	iv
Abstract	v
1. Introduction	1
2. Background	6
2.1. Task Definition and Terminology	6
2.1.1. Failure, Fault, and Repair-Location Labels	6
2.1.2. Boundaries with Issue Localization, Repair, and Patch Assessment	7
2.2. Evidence Regime	7
2.3. LLM Inference Engines	8
2.3.1. Role and Serving Workflow	8
2.3.2. Repository Heterogeneity	8
2.3.3. Symptom–Repair Distance	9
2.3.4. Limited and Uneven Diagnostic Evidence	9
2.4. Summary	9
3. Related Work and Thesis Positioning	10
3.1. Execution-Dependent Fault Localization	10
3.2. Issue-Guided Retrieval and Learned Ranking	11
3.3. Repository-Scale LLM Exploration	13
3.4. Localization in Complex Systems Repositories	14
3.5. Inference-Engine Reliability and Diagnosis	15
3.6. Datasets and Evaluation Constructs	15
3.7. Synthesis and Thesis Positioning	15
3.7.1. Evaluation Gap	15
3.7.2. Design Gap and Bounded Novelty Claim	16
4. Evaluation of LLM-Agent-Based Fault Localization on Ollama Bugs	18
4.1. Dataset Construction	18
4.1.1. Candidate Issue–PR Mining and Fix Confirmation	19
4.1.2. Pre-Fix Snapshot Reconstruction and Raw Change Extraction . .	19

4.1.3.	Repair-Location Cleaning and Final Label Construction	20
4.1.4.	Candidate Universe and Evaluation Inputs	20
4.1.5.	Final Dataset	21
4.2.	Study Setup	21
4.2.1.	Studied Baselines	21
4.2.2.	Backbone Model Selection	23
4.2.3.	Implementation and Reproducibility	24
4.3.	Evaluation Metrics	25
4.3.1.	Effectiveness Metrics	26
4.3.2.	Efficiency Metrics	26
4.4.	Experimental Results	27
4.5.	Quantitative Analysis	29
4.5.1.	Comparison with IR-based approaches	29
4.5.2.	Uniqueness and Union	30
4.5.3.	Comparison on Different Artifact-Type	31
4.5.4.	Comparison on Computational Cost	32
4.6.	Qualitative Analysis	33
4.6.1.	Evidence Reachability Failures	33
4.6.2.	Causal-Chain Truncation	34
4.6.3.	Non-Source Artifact Underinspection	35
4.7.	Summary	35
5.	CLUEFL: Cross-Software-Layer Understanding of Evidence for Fault Localization	37
5.1.	Approach	38
5.1.1.	Overview	38
5.1.2.	Hypothesis Generation	38
5.1.3.	Evidence Exploration	41
5.1.4.	Relevance Ranking	43
5.2.	Experimental Setup	43
5.3.	Experimental Results	44
5.4.	Quantitative Analysis	45
5.4.1.	Uniqueness and Union	45
5.4.2.	Comparison on Different Artifact Type	45
5.4.3.	Component Ablation	46
5.5.	Qualitative Analysis	48
5.5.1.	Impact of Project-Specific Layer Knowledge	48
5.5.2.	Impact of Historical Issue Retrieval	49
5.5.3.	Impact of Root-Cause Expansion	50

5.5.4. Impact of Repository Navigation	50
5.6. Summary	51
6. Discussion and Conclusion	52
6.1. Threats to Validity	52
6.2. Future Research	53
6.3. Summary	54
A. Supplementary Materials	55
A.1. Agent Prompts	55
A.1.1. Baseline SWE-agent Prompt	55
A.1.2. CLUEFL Prompt	57
A.2. Ollama Software-Layer Taxonomy	60
Abbreviations	61
List of Figures	62
List of Tables	63
Bibliography	64

1. Introduction

Large language models (LLMs) are increasingly deployed in interactive applications, developer tools, and local artificial intelligence services. Their practical use depends on systems software that executes pretrained models and mediates among model representations, heterogeneous hardware, and downstream applications. Following prior work, this thesis refers to this repository-scale software as an *LLM inference engine*: beyond model execution, it includes the request-serving, model-management, resource-allocation, backend-selection, configuration, build, and deployment components required to deliver inference functionality [Liu+26b]. Faults in this infrastructure can affect the availability, correctness, and security of many downstream applications [Liu+26b; Zha+26c].

An LLM inference engine is not a self-contained model-execution module. It coordinates request processing, model loading, scheduling, resource management, native execution, hardware adaptation, and build and deployment support while accommodating changing model architectures, operating systems, accelerators, drivers, model formats, and configurations [Liu+26b; LW26]. These responsibilities are tightly coupled: a symptom observed through an application programming interface (API) or a command-line interface (CLI) may originate in a different functional region and be repaired in a different type of repository artifact.

Existing research addresses adjacent but distinct diagnostic tasks. Empirical studies characterize the symptoms, root causes, and repair patterns of inference-engine bugs [Liu+26b]; static issue-centric approaches predict coarse root-cause or module categories [LW26]; and runtime approaches diagnose reproducible failures at the state or component level using intermediate states, traces, metrics, or fault injection [Gu+26; Sch+26]. Although these studies provide valuable diagnostic knowledge, they do not rank the concrete files that developers modify in the corresponding pre-fix repository. This leaves a distinct fault-localization problem unaddressed. Given an issue report and its corresponding pre-fix repository snapshot, a localizer must identify the concrete file that developers subsequently modify to repair the reported bug.

To the best of our knowledge, no prior work has formulated and systematically evaluated this issue-guided, execution-free file-level fault-localization task for bugs in LLM inference-engine software. Accordingly, it remains unknown how effectively existing localization approaches can address this problem.

Fault-localization approaches rely on different evidence sources [Won+16]. Execution-based approaches derive suspiciousness from passing and failing executions but cannot be applied consistently here: historical revisions may require models, hardware, configurations, or runtime states that are unavailable, while silent or nondeterministic failures may lack a reliable oracle. Information-retrieval-based fault localization (IRFL) remains applicable because it ranks repository files based on their correspondence to a bug report [ZZL12; Sah+13]. Directly prompted LLMs can reason beyond surface vocabulary, but preselected code may exclude the repair file and full-repository prompting is impractical under context and cost constraints; direct reasoning therefore does not solve candidate discovery [Wu+23]. In recent studies, it has been found that LLM agents can instead search a repository iteratively, though generic access to tools does not ensure that they follow the causal path from symptom to repair location [KAY24; Zho+26]. These considerations motivate evaluating IR-based and LLM-agent-based approaches in a common execution-free setting.

This thesis studies an explicitly *execution-free* setting. A localizer receives a cleaned issue report and its pre-fix repository snapshot, but does not compile the repository, run tests or models, or collect new runtime traces. Existing logs and stack traces in the report remain admissible. The setting targets bugs for which reproducible execution evidence is unavailable or prohibitively expensive; it does not imply that execution-based localization is generally unsuitable.

Moreover, the fault localization in LLM inference engines is especially challenging for three reasons:

- **Repository scale and artifact heterogeneity.** At the Ollama snapshot closest to the dataset-collection cutoff, the repository contained 1,254 tracked files and approximately 324,000 lines of implementation, build, and template code, including about 238,000 lines of Go, 69,000 lines of C/C++ and header code, and 10,000 lines of TypeScript.¹ Relevant logic is distributed not only across these languages but also across source and non-source artifacts, including configuration files, build specifications, workflows, templates, and model metadata [Oll26a]. This produces a large and heterogeneous candidate space that cannot be inspected exhaustively.
- **Architectural coupling and symptom–repair distance.** An issue describes where a failure is observed, not necessarily where the responsible implementation resides. Prior work identifies 28 root causes spanning input/output handling, configuration, functionality, environment, and resource management, and shows

¹Measured with `cloc` 2.11 at Ollama commit `2ea95fb` from 30 June 2026. Large vocabulary files and textual data in formats such as Markdown, JavaScript Object Notation (JSON), and YAML Ain’t Markup Language (YAML) were excluded from the total source lines of code. Individual dataset instances use their corresponding historical pre-fix snapshots and therefore vary slightly in size.

that similar symptoms may arise from different causes [Liu+26b]. An API-visible symptom may thus require a repair in scheduling, model metadata, runtime code, hardware adaptation, or deployment logic elsewhere. Report and repair-file vocabulary may differ substantially, especially when they lie across software layers.

- **Limited diagnostic evidence.** Issue reports do not necessarily provide an executable failing test, complete reproduction conditions, logs, or a stack trace. Reproduction may additionally depend on a particular model, accelerator, driver, operating system, memory capacity, configuration, or transient state. Unexpected-output, nondeterministic, and silent failures may offer neither an explicit trace nor a reliable pass/fail oracle [Liu+26b; Gu+26]. Similar observability limitations occur in issue-guided Linux kernel localization [Zho+26]. A localizer must therefore use whatever diagnostic clues the report provides.

To address the lack of systematic research on issue-guided, execution-free file localization for LLM inference-engine bugs, this thesis is guided by the following research questions (RQs):

- **RQ1 – Feasibility Exploration:** Despite the aforementioned challenges, are representative IR-based and LLM-agent-based approaches feasible for execution-free file-level localization of the issues in the LLM inference engines?
- **RQ2 – Comparative Analysis:** What trade-offs do these approaches exhibit in ranking effectiveness, underlying language model, and computational cost?
- **RQ3 – Failure Explainability:** Do LLM-agent localization failures in LLM inference engines reduce to a small set of recurring, identifiable pitfalls?
- **RQ4 – Domain-Specific Improvement:** Can incorporating domain-specific knowledge and a well-designed workflow improve fault localization in LLM inference engines?

We select Ollama as a representative case study of modern LLM inference software because it combines broad practical adoption with coverage of the principal architectural concerns found across inference engines. As of July 2026, Ollama reported serving 8.9 million developers [Oll26b]. Its multi-language codebase integrates model acquisition and management, API serving and request orchestration, resource-aware scheduling, inference runners, hardware-specific backends, and platform-specific build and deployment artifacts [Oll26a]. These elements correspond to the common inference-engine workflow of engine setup, model conversion, and inference/serving, as well

as the configuration, functionality, environment, and resource-management concerns observed across widely adopted engines [Liu+26b]. Consequently, Ollama captures a substantial portion of the software-layer heterogeneity characteristic of LLM inference engines, providing a practically grounded and architecturally broad setting for evaluating cross-software-layer file localization.

For evaluation, We construct a fault-localization dataset from validated Ollama issue-fix pairs collected up to 1 July 2026. The dataset retains only fixes that modify exactly one eligible file, so the developer-modified file provides an unambiguous repair-location label. All approaches operate on the corresponding pre-fix repository snapshots to prevent information leakage. We use BugLocator and BLUiR as representative IR-based approaches, emphasizing historical bug similarity and structured code retrieval, respectively. As a representative LLM-agent-based approach, we select SWE-agent because it is an established open-source framework for repository-level issue resolution that enables iterative repository inspection through a structured agent-computer interface [Yan+24b]. SWE-agent also records structured interaction trajectories, including agent actions and environment observations, allowing us to analyze its search behavior and recurring failure modes. Its domain-agnostic design makes it an appropriate generic baseline for assessing the contribution of inference-engine-specific guidance.

As for results, the IR experiments reveal useful but imprecise candidate reduction. On the single-file dataset, BugLocator and BLUiR rank fewer than one quarter of repair files first (23.45% and 21.38%, respectively), while approximately half appear within the Top-5 (48.97% and 52.41%): the repair file is often retrieved but not prioritized early. Their overall performance is close and cutoff-dependent. BugLocator is generally stronger at Top-1, Top-3, and mean reciprocal rank at five (MRR@5), whereas BLUiR leads only at Top-5. Case analysis shows complementary evidence: BugLocator benefits from analogous historical reports, BLUiR from issue terms aligned with code identifiers, and both from direct lexical cues. Both struggle when sparse reports or symptom-repair distance remove surface overlap. These results show that IRFL remains useful for candidate reduction even in a large, complex, and heterogeneous repository, but cannot reliably prioritize the repair location.

The agent experiments show a different pattern. Across the 145 single-file cases, both SWE-agent configurations already place the majority of repair files first and the large majority within the Top-5, producing substantially more top-heavy rankings than the IR approaches: 51.03% Top-1 / 75.17% Top-5 for GPT-5.2, and 64.83% Top-1 / 88.97% Top-5 for DeepSeek V4 Pro. Under matched backbone conditions, the Cross-Software-Layer Understanding of Evidence for Fault Localization (CLUEFL) framework further improves early prioritization on both backbones: on GPT-5.2, it raises Top-1 by 7.59 percentage points (74 to 85 of 145 cases), Top-5 by 6.21 percentage points (109 to 118 of 145 cases), and MRR@5 by 0.0629; on DeepSeek V4 Pro, it raises Top-1 by 2.76 percentage points

(94 to 98 of 145 cases) and MRR@5 by 0.0195, while Top-5 coverage is unchanged. The more limited gain for the DeepSeek V4 Pro configuration coincides with its higher observed SWE-agent baseline scores, which leave less room for improvement; the largest observed benefit therefore occurs for the GPT-5.2 configuration, whose baseline scores are lower.

Lastly, we summarize the contributions of this thesis as follows:

- **The first systematic evaluation of file localization for LLM inference-engine bugs.** To the best of our knowledge, this thesis is the first to evaluate issue-guided, execution-free file localization in an LLM inference engine, comparing representative IR-based and LLM-agent-based approaches in effectiveness, cost, backbone models, and failure behavior.
- **A repository-derived evaluation dataset.** We publicly release 145 validated Ollama issue–fix pairs with corresponding pre-fix snapshots and unambiguous, patch-derived single-file repair labels spanning eligible source and non-source artifacts. The dataset provides a reusable foundation for future research on file localization and debugging support for LLM inference-engine software.
- **A domain-specific localization framework.** We propose CLUEFL as an initial step toward domain-specific localization and provide evidence that external knowledge and a well-designed workflow can improve file-ranking effectiveness for LLM inference-engine bugs.

Chapter 2 presents the background. Chapter 3 reviews related work and positions this thesis. Chapter 4 describes the dataset and evaluates existing localizers. Chapter 5 presents and evaluates CLUEFL. Chapter 6 discusses threats to validity and directions for future research before concluding the thesis.

2. Background

This chapter defines the task, the admissible evidence, and the inference-engine concepts needed by the remainder of the thesis. It deliberately does not survey competing localization approaches. That comparison is deferred to Chapter 3, where differences in evidence requirements, output granularity, and target scope can be made explicit.

2.1. Task Definition and Terminology

2.1.1. Failure, Fault, and Repair-Location Labels

A *failure* is an externally observed deviation from expected behaviour, whereas a *fault* is a program element or state capable of causing that failure. Fault localization (FL) aims to identify program elements associated with a fault and commonly returns a ranked list of suspicious files, methods, or statements [Won+16]. The task studied in this thesis is:

"Given a cleaned issue report and the corresponding pre-fix repository snapshot, rank the eligible repository files so that the developer-modified repair file appears near the top of the ranking."

The exact causal fault is rarely observable in a repository-mining study. Instead, files modified by the accepted developer fix provide observable *repair-location labels*. Such labels do not prove that every changed file independently contained a root cause, or that no alternative repair would have been valid. This thesis therefore uses *repair file* when interpreting a label and retains *ground truth* only in formal metric definitions. To avoid ambiguity in set-valued labels, effectiveness analyses use single-target instances whose accepted fix modifies exactly one eligible repair file. The dataset-construction chapter defines the corresponding inclusion rules.

The output is a repository-level, file-granularity ranked list. Here, repository-level describes the search scope rather than requiring the output to enumerate every repository file: a method may omit candidates assigned a score of zero. The candidate universe is intentionally not restricted to conventional source code: an eligible build specification, workflow, configuration, template, or model metadata file may be returned when that

artifact is maintained in the target repository. The dataset and evaluation methodology define file eligibility, snapshot construction, and ranking metrics.

2.1.2. Boundaries with Issue Localization, Repair, and Patch Assessment

Repository issue localization is sometimes used as a broader term for finding the code entities relevant to a bug report, feature request, performance issue, or security issue. In contrast, this thesis selects reports classified as bugs and evaluates their observed repair locations. Automated program repair (APR) must additionally synthesize a change, while repository-level issue resolution commonly judges a generated patch with executable tests. SWE-bench, for example, pairs an issue with a repository but evaluates a test-validated patch rather than an independent localization ranking [Jim+24]. Repair success therefore combines at least two capabilities: finding a useful location and constructing an acceptable change.

Automated patch-correctness assessment (APCA) starts even later in the pipeline: it estimates whether an already generated patch is semantically correct. Learned assessors such as APPT and RePaCA are relevant to the reliability of repair pipelines, but they neither retrieve a repair file from a repository nor measure an execution-free file ranking [Zha+24; Fus+26]. These task boundaries prevent repair or patch-classification accuracy from being reported as localization accuracy.

2.2. Evidence Regime

In this thesis, *execution-free* means that the localizer does not compile the target revision, run the engine or its tests, instrument the program, inject faults, or collect new runtime traces. The localizer receives only the cleaned issue report and the pre-fix repository snapshot. The issue may contain logs, measurements, reproduction steps, or an exception trace supplied by the reporter. These are historical artifacts already present at reporting time; they are not dynamic signals generated by the localization approach.

This definition is stricter than *coverage-free* and more precise than *test-free*. A coverage-free approach may still require a failed-test output, crash dump, execution trace, or reference run. Conversely, an execution-free approach may read a stack trace embedded in the issue without executing the old revision. The distinction matters because approaches described as “bug-report based” can have materially different evidence requirements. It also rules out quietly supplementing a sparse historical report with signals that were not available in the controlled experimental setting.

The case-specific evidence consists of a cleaned issue report and the corresponding pre-fix repository snapshot. Each approach receives the same version of these inputs, making the ranking task comparable across classical retrieval and LLM-based

approaches. Method-specific static knowledge and retrieval resources are documented separately.

2.3. LLM Inference Engines

2.3.1. Role and Serving Workflow

An LLM inference engine is systems software that executes pretrained models and mediates among model representations, heterogeneous hardware, and downstream applications. Besides model execution, it includes request serving, model management, resource allocation, backend selection, configuration, build, and deployment components [Liu+26b]. During serving, *prefill* processes the prompt and constructs attention state; repeated *decode* iterations then generate tokens. Their different computation and latency profiles create scheduling and memory-management trade-offs [Agr+24].

Representative engines emphasize different parts of this workflow. Orca introduced iteration-level scheduling and selective batching [Yu+22]. vLLM introduced PagedAttention and block-based management of the key-value cache [Kwo+23]. Sarathi-Serve combines chunked prefill with stall-free scheduling [Agr+24], while SGLang combines a language-level frontend with an optimized runtime and cache reuse [Zhe+24]. Ollama, the empirical subject of this thesis, combines local model management, command-line and Hypertext Transfer Protocol (HTTP) interfaces, model metadata and templates, Go and native inference paths, hardware discovery, and deployment tooling [Oll26a]. It is not assumed to be architecturally equivalent to a distributed datacentre engine such as vLLM or SGLang.

2.3.2. Repository Heterogeneity

Inference behaviour emerges from interactions among public interfaces, request orchestration, model representation, scheduling and cache policy, native kernels, hardware backends, and build or deployment artifacts. A repository-level localizer must consequently search across languages and artifact types. In Ollama, source files coexist with model templates, build rules, dependency manifests, workflows, and configuration. Restricting the candidate universe to a single language or extension would encode a location answer before localization begins.

The unit called a *software layer* later in this thesis is an operational grouping of repository responsibilities, not a neural-network layer. The repository-specific layer inventory and diagnostic checklist are contributions of CLUEFL; they are therefore introduced and operationalized in the framework chapter rather than presented here as established background.

2.3.3. Symptom–Repair Distance

An issue is written at the level at which a failure becomes visible, but its repair may lie elsewhere. An API error can originate in model metadata, runner selection, a native runtime, a hardware backend, or a build rule. The words in the report can therefore have little overlap with the responsible file. This thesis calls the resulting separation in vocabulary and software responsibility *cross-software-layer semantic distance*.

Empirical evidence supports treating this distance as a domain characteristic. The study by Liu et al. analyses 929 bugs across five inference engines and reports root causes spanning input/output processing, configuration, functionality, environment, and resource management [Liu+26b]. Li and Wang likewise describe tightly coupled software and hardware modules in vLLM and TensorRT-LLM [LW26]. These published classifications motivate attention to cross-component failures, but they are not copied into the present chapter as a localization procedure.

2.3.4. Limited and Uneven Diagnostic Evidence

Historical issues do not consistently provide a failing test, complete reproduction environment, log, or stack trace. Reproduction may require a specific model, accelerator, driver, operating system, memory capacity, configuration, or transient runtime state. Silent and nondeterministic failures may offer no reliable pass/fail oracle [Liu+26b; Gu+26]. Consequently, the amount and specificity of admissible evidence vary substantially across reports. Execution-free localization does not solve this scarcity; it studies what can be inferred under it.

2.4. Summary

The thesis evaluates a temporally controlled, execution-free ranking of repair files in a heterogeneous inference-engine repository. Its labels are observed developer repair locations rather than uniquely proved causes. The domain is challenging because a visible symptom may be semantically and architecturally distant from a repair in source or non-source artifacts. The next chapter asks which parts of this setting are already addressed by prior localization, repository-agent, benchmark, and inference-engine-reliability research.

3. Related Work and Thesis Positioning

This review covers publications available by 17 August 2026. Peer-reviewed versions are cited where available; recent preprints are treated as provisional evidence. Methods are grouped by the evidence they require and the output they evaluate, not merely by whether they contain an LLM. This organization avoids comparing a file ranker with an approach that has already been given a failing test, a source file, or an executable repair environment.

3.1. Execution-Dependent Fault Localization

Spectrum-based fault localization instruments passing and failing executions and derives suspiciousness from their coverage. Mutation-based fault localization observes how injected variants change test outcomes; Metallaxis-FL is a representative technique [PT15]. Such approaches can produce fine-grained rankings when a reproducible failure, an adequate test suite, and a meaningful oracle are available [Won+16].

LLM agents can organize the same dynamic evidence. AutoFL uses tool calls to inspect test-covered classes and methods [KAY24]. AgentFL stages fault comprehension, navigation, and confirmation [Qin+24]; its peer-reviewed successor SoapFL uses failing-test behaviour and coverage in a method-level procedure [Qin+25b]. FlexFL accepts reports, tests, or both, obtains a small method candidate set, and applies LLM refinement [Xu+25]. MemFL combines a static project summary with a dynamic memory formed from prior test-based localization attempts [YRB25]. SieveFL applies dense file retrieval, JaCoCo execution filtering, isolated method screening, and comparative reranking with an open model served through Ollama [FG26]. Its local deployment is relevant to this thesis, but its central pruning signal is newly collected runtime coverage. These works establish the value of structured reasoning, but their primary configurations do not match a report-only file-ranking task.

Other approaches turn a specific runtime artifact into a semantic bridge. FaR-Loc uses a failed test and stack trace to generate a functionality description, retrieves among covered methods, and reranks them with an LLM [SLC25]. T2L-Agent narrows vulnerability localization from repository regions to lines using crash points, stack traces, and abstract-syntax-tree (AST) chunks [Xi+26]. AutoCrashFL starts from a crash dump and source repository, and is evaluated on industrial SAP HANA crashes rather

than issue-only bugs [Kan+25]. IssueExec retrieves tests as executable proxies for requirements and executes them to build trace graphs [Liu+26a]. Its study reports that existing tests reach most repair files in its Python sample, but its own threats explicitly exclude non-runnable or test-inaccessible issues. These are strong dynamic localizers, not matched execution-free baselines.

SemLoc takes a different route to verifiable evidence: it converts LLM-inferred semantic properties into typed executable constraints, evaluates their violations across passing and failing tests, and applies counterfactual verification [Yan+26]. Its repository-scale study is a two-stage feasibility analysis in which AutoFL first supplies a target function and SemLoc refines it to lines. The approach therefore supports the value of structured, checkable semantic evidence, but it does not solve the report-only repository-file search studied here.

Project history can also modify an existing suspiciousness signal. PAFL mines recurring statement patterns from earlier buggy project versions and uses them to post-process several baseline localizers [Kim+25]. It establishes project-specific fault patterns as prior art, while differing from this thesis in granularity, baseline evidence, and the availability of labelled historical faults.

Recent domain-specific systems reinforce the boundary. CoHiKer localizes Linux kernel faults with generated passing/failing tests, crash evidence, system-call context, and dependency information [Wan+26]. EffiHolmes localizes repository-level time inefficiencies from differential profiling and critical execution paths [YPD26]. Ekka compares intermediate states of an optimized inference engine with a reference implementation to diagnose silent output errors [Gu+26]. Their runtime evidence is powerful when obtainable; it cannot be reconstructed fairly for every historical Ollama issue in the dataset used here.

3.2. Issue-Guided Retrieval and Learned Ranking

Classical information-retrieval-based bug localization (IRBL) treats the report as a query and source files as documents. BugLocator combines textual similarity with previously fixed similar bugs [ZZL12], while BLUiR indexes structured report and code fields [Sah+13]. Bench4BL showed that revision matching and dataset construction materially affect reported performance [Lee+18]; FinerBench4BL transforms the file-level benchmark into a method-level evaluation framework and reports the resulting accuracy–inspection-effort trade-off [THA23]. The family is inexpensive and transparent, but lexical retrieval is vulnerable to symptom–repair vocabulary mismatch and is often limited to source files [Niu+25].

IncBL addresses operational scalability rather than a new relevance model: it updates

BugLocator-style retrieval incrementally as reports and repositories evolve, reducing recomputation while preserving the original ranking accuracy [Yan+21]. Faster model maintenance is therefore not evidence of better semantic localization.

LLMs can intervene without removing the explicit retrieval stage. BRaIn uses LLM-generated relevance feedback for reformulation and reranking [SR25]; LLM-BL expands the report, compresses retrieved code, and performs zero-shot listwise reranking [Li+25]. GenLoc combines semantic retrieval with iterative code exploration and evaluates file localization on Java issue data, GHRB, and SWE-bench Lite [AYM26]. SweRank learns issue-conditioned retrieval and reranking from issue-change pairs [Red+26]. HyperFL further adapts the issue representation itself by generating query-specific low-rank adaptation parameters while keeping code embeddings reusable [Sha+26b]. Its evaluation reinforces that issue reports vary substantially in structure and diagnostic content, although its learned corpus and target universe remain source-code files and functions. These systems make candidate generation visible, which is valuable for diagnosing a miss.

More specialized hybrids clarify what the LLM contributes. LLmiRQ selects query strategies from report content and adds conversational reformulation and learning-to-rank features [SY24]. CosFL asks an LLM to describe the functionality of potentially faulty methods, then performs multi-granular semantic code search [Qin+25a]. LLMLoc combines semantic retrieval with structural embeddings for zero-shot function localization [NY25]. LLMAO instead adapts pretrained representations for coverage-free line ranking inside an already bounded source file [Yan+24a]. The first three improve retrieval or ranking over a defined code corpus; LLMAO begins after repository-level file selection. None demonstrates discovery of eligible configuration, build, workflow, or metadata targets.

The 2026 literature shifts attention from the ranking model alone to the representation retrieved. A controlled study of five repository representations finds that role-aware summaries can retain strong localization utility at a fraction of raw-code context [CCC26]. Reformulate-Retrieve-Localize uses an LLM-refined query before Best Matching 25 (BM25) retrieval and then an agentic selection stage [CM26]. BLAgent uses AST- and path-aware encoding, dual query transformation, compact retrieval, and bounded agentic reranking [MU26]. Its strong SWE-bench Lite results show that structured retrieval plus reranking is already an established design; it does not, however, evaluate inference-engine-specific artifact relations or the same source/non-source candidate universe as this thesis.

3.3. Repository-Scale LLM Exploration

General issue-resolution agents search a repository while attempting a patch. SWE-agent introduced an agent–computer interface for repository interaction [Yan+24b]; OpenHands provides a broader platform for software development agents [Wan+24]; and Agentless separates repository structure, file retrieval, location selection, patch generation, and validation [Xia+24]. These systems demonstrate repository-scale search, but end-to-end repair scores do not isolate localization quality. This thesis therefore evaluates a localization-only adaptation of a lightweight SWE-agent interface under a matched evidence protocol [SWE26].

FL-specific agents add stronger repository representations. LocAgent builds a heterogeneous graph of directory, file, class, and function entities and gives an LLM graph-navigation tools [Che+25]. CoSIL dynamically builds and prunes module and function-call relations [Jia+25]. OrcaLoca combines action decomposition, priority scheduling, and distance-aware context pruning [Yu+25]. RepoNavigator shows that a learned agent can navigate with a highly constrained jump-to-definition action [Zha+26b]. These approaches reduce indiscriminate context loading, although symbol-centric navigation can make a configuration or build artifact unreachable when it is not represented as a code entity.

ARISE extends this graph line of work to statement nodes and intra-procedural definition–use edges, exposing data-flow slices as on-demand agent tools [Sed+26]. On SWE-bench Lite it improves function- and line-level localization over its host agent, while increasing rather than reducing total token use and remaining limited to Python source entities. It is direct prior art for queryable program-analysis primitives, but not for heterogeneous execution-free file ranking.

BugCerberus uses separate models and representations to narrow localization from files to functions and statements [Cha+25]. SeeRepo takes a different representational route: it renders repository structure as a visual graph while retaining source text for precise inspection [Ma+26]. Its SWE-bench Verified experiments suggest that visual structure is most useful during localization rather than repair or validation, but the evidence is Python-only and its end task remains issue resolution.

GraphLocator explicitly models the path from reported symptom to potential cause as a dynamically constructed issue graph [Liu+26c]. RGFL adds per-file natural-language reasoning and embedding/LLM ranking after Agentless-style retrieval [Sep+26]. HiLoRM instead trains a hierarchical reward model to select among sampled file-, function-, and line-level predictions [Zha+25]. Together, these works mean that graph navigation, causal framing, hierarchical localization, and bounded evidence inspection are not novelty claims available to this thesis.

Granularity studies further warn against treating a narrower location as automatically

better context. A factorial study on SWE-bench Verified reports that file-level context dominates downstream repair performance, while broad line-level expansion can introduce harmful noise [SPH26]. Because that study supplies ground-truth locations to isolate patch generation, it is an optimistic context-selection result rather than evidence that a practical file localizer found those files.

Two adjacent benchmarks broaden the notion of evidence and artifact. LIDL builds a knowledge graph over source code, prompts, and configurations for applications that integrate LLMs and combines report, inferred, and available runtime evidence [Tan+26]. MM-IssueLoc contributes 652 issue–pull request pairs with controlled visual evidence and evaluates file and function localization across 23 languages [Zha+26a]. LIDL establishes heterogeneous target awareness, while MM-IssueLoc establishes multimodal issue evidence; neither is an execution-free study of bugs inside an inference engine. The latter is also outside this thesis’s text-only cleaning protocol, but it identifies a clear extension for issues whose screenshots carry the only visible symptom.

3.4. Localization in Complex Systems Repositories

LinuxFLBench and LinuxFL+ are the closest system-software benchmarks [Zho+26]. They curate 250 Linux-kernel tasks and evaluate file- and method-level localization; LinuxFL+ adds directory awareness and expands issue evidence with potential causes and mailing-list information. Their analyses expose incomplete exploration and confusion among related files, which are directly relevant failure modes. The comparison is nevertheless not matched: Linux kernel tasks are C-oriented and their history enrichment does not correspond to the temporally restricted, issue-only evidence used here. CoHiKer is still more evidence-rich because it generates contrasting test executions [Wan+26].

ConFL is a closer execution-free domain-aware neighbour: from a bug report it constructs a Java concurrency knowledge base, narrows the search hierarchically, and ranks suspicious methods using an interaction-level domain-specific language [Sha+26a]. Its ISSTA 2026 evaluation shows that structured domain knowledge and report-only hierarchical reasoning are already established in a specialized FL setting. The remaining distinction is consequently not the use of domain guidance itself, but the inference-engine operational model, the source/non-source file universe, and the matched historical protocol.

These systems studies and the dynamic approaches in Section 3.1 jointly show why “domain-aware” is not itself a novelty claim. The differentiating question is whether domain guidance remains useful when no test execution, trace, crash dump, or coverage-derived candidate set may be generated.

3.5. Inference-Engine Reliability and Diagnosis

Liu et al. provide the most comprehensive empirical foundation for this domain. Their study manually analyses 929 bugs from Llama.cpp, vLLM, DeepSpeed, MLC-LLM, and TensorRT-LLM, deriving symptom, root-cause, component, and repair classifications [Liu+26b]. The work explains why inference-engine faults cross serving and deployment responsibilities, but it does not evaluate an approach that ranks files for an unseen issue. Ollama is also not one of the five mined engines.

Li and Wang study static issue-centric reliability analysis for vLLM and TensorRT-LLM [LW26]. Their pipeline classifies issue-level causes, maps reports to predefined modules through textual similarity, and asks an LLM for repair suggestions. This is useful triage, but its predicted location is a coarse module rather than a repository-file ranking, and the suggestion is not grounded in inspection of the corresponding pre-fix implementation. Runtime studies such as Ekka or telemetry-based root-cause analysis answer a complementary question with traces, reference executions, metrics, or fault injection [Gu+26; Sch+26].

Thus, prior inference-engine work supplies empirically grounded domain concepts and diagnostic categories. It does not establish comparative file-ranking performance for classical retrieval, a generic repository agent, and a domain-guided agent on the same execution-free historical issues.

3.6. Datasets and Evaluation Constructs

The benchmark choice determines what a reported score means. Defects4J supports controlled execution; Bench4BL and FinerBench4BL support report-to-file retrieval; SWE-bench evaluates test-validated repair; LocBench and MM-IssueLoc broaden repository and evidence settings; LinuxFLBench tests complex systems software. None is interchangeable with an Ollama-specific, temporally controlled, execution-free candidate ranking. The dataset created here is a domain-specific instrument rather than a replacement for these resources.

3.7. Synthesis and Thesis Positioning

3.7.1. Evaluation Gap

The literature does not establish how classical IRBL and repository agents behave on the same set of historical inference-engine bugs under a matched, execution-free protocol. In particular, it does not quantify, for this domain, how often non-source targets are

Table 3.1.: Representative benchmarks. Patch-derived labels are observed developer repair locations, not uniquely proved causal locations.

Benchmark	Primary evidence/task	Reference output	Boundary for this thesis
Defects4J [JJE14]	Reproducible Java bugs and tests	Triggering tests and curated patches	Execution-centred; not an issue-only inference-engine setting
Bench4BL / FinerBench4BL [Lee+18; THA23]	Bug-report-to-Java-file/method retrieval	Patch-derived source files or transformed methods	Direct IRBL precedent, but source-only and not systems software
SWE-bench [Jim+24]	Repository issue resolution in executable environments	Test-validated patch	End-to-end repair, not an isolated file ranking
LocBench [Che+25]	Issue-guided Python localization	Patch-derived files, classes, functions	General repositories and source-code entities
LinuxFLBench [Zho+26]	Linux-kernel localization	Patch-derived files and methods	Closest systems analogue; different language, history, and artifact regime
MM-IssueLoc [Zha+26a]	Multimodal issue localization	Patch-derived files and functions	Images are controlled inputs; report types and repositories are broad
Dataset in this thesis	Cleaned report plus pre-fix Ollama snapshot	Eligible single repair file	One domain; source and eligible non-source artifacts; no new execution

missed, how results change with the backbone model, or what accuracy–cost trade-off arises when lexical retrieval is replaced by iterative inspection. The dataset and baseline study address this *evaluation gap*.

3.7.2. Design Gap and Bounded Novelty Claim

The design gap is not that LLMs have never used repository context. LocAgent, CoSIL, OrcaLoca, GenLoc, BLAgent, HyperFL, GraphLocator, ARISE, RepoNavigator, RGFL, ConFL, and LIDL already demonstrate adaptive retrieval, graph traversal, iterative exploration, causal framing, domain-specific reasoning, bounded reranking, and heterogeneous artifacts. Nor are empirical bug taxonomies for inference engines new [Liu+26b].

The narrower untested combination is an execution-free file localizer that uses an inference-engine-specific operational model to inspect a temporally correct snapshot while retaining eligible source and non-source targets. The review found adjacent work covering each ingredient separately, but no study evaluating this combination on historical bugs inside an inference engine under a matched file-ranking protocol. CLUEFL addresses this gap with an analysis-oriented software-layer map, typed artifact relations, explicit hypothesis generation, evidence verification, and bounded ranking.

3. Related Work and Thesis Positioning

Table 3.2.: Positioning by primary evaluated configuration.

Approach	Primary evidence	New execution	Output	Main target scope
BugLocator / BLUiR [ZZL12; Sah+13]	Report, source text, optional history	No	File ranking	Source files
LLM-BL / CosFL / BLAgent / HyperFL [Li+25; Qin+25a; MU26; Sha+26b]	Report and retrieved repository content	No required trace	File or method ranking	Primarily source code
AutoFL / SoapFL / MemFL [KAY24; Qin+25b; YRB25]	Tests, failure output, coverage or dynamic memory, plus code	Yes in the primary workflow	Method ranking	Source methods
IssueExec / FaR-Loc / T2L-Agent [Liu+26a; SLC25; Xi+26]	Tests, traces, crash or stack evidence, plus code	Yes	File/function/line ranking	Source-code entities reached by runtime evidence
SWE-agent adaptation	Cleaned issue and pre-fix repository	No	File ranking	Eligible source and non-source files
LocAgent / GraphLocator / ARISE	Issue, code graph, retrieved entities	No required trace	File/function ranking	Code entities and relations
LIDL	Report, heterogeneous artifacts, available error evidence	No new execution reported	Multi-file ranking	Source, prompt, config
CLUEFL	Cleaned issue and pre-fix inference-engine repository	No	File ranking	Eligible source and non-source files

The repository-specific map and diagnostic checklist are framework artifacts and are introduced in Chapter 5, not claimed as general background.

4. Evaluation of LLM-Agent-Based Fault Localization on Ollama Bugs

This chapter constructs the empirical basis for issue-guided, execution-free file-level fault localization analysis in Ollama. It first describes the construction of the Ollama evaluation dataset and then evaluates three representative localization approaches: BugLocator [ZZL12], BLUiR [Sah+13], and a localization-only adaptation of SWE-agent [Yan+24b; SWE26]. BugLocator and BLUiR represent two established IR-based fault-localization (IRFL) designs, whereas SWE-agent represents iterative, LLM-guided repository exploration.

The evidence boundary follows Chapter 2. Each approach receives a cleaned issue report and the corresponding pre-fix repository snapshot. The approaches do not compile or execute the target revision, run its tests or models, or collect new runtime evidence. The task is to rank the eligible repository files so that the developer-modified repair file appears near the top.

This chapter addresses the following three research questions introduced in Chapter 1. The chapter is organized as follows. Section 4.1 describes the construction of the Ollama evaluation dataset. Section 4.2 defines the evaluated approaches and common protocol. Section 4.3 introduces the metrics. Section 4.4 presents the quantitative results and answers RQ1, Section 4.5 interprets those results in greater depth and answers RQ2, Section 4.6 analyzes the trajectories and answers RQ3, and Section 4.7 summarizes the answers to the research questions.

4.1. Dataset Construction

The *Ollama evaluation dataset* is mined from the public histories of issues and pull requests (PRs) in the ollama/ollama repository up to 1 July 2026. An instance consists of (i) a GitHub issue report, (ii) the corresponding pre-fix repository snapshot, and (iii) the eligible repository file modified in the accepted fixing change by developers. The modified file is treated as the observable *repair-location label*, or *ground truth*. Figure 4.1 summarizes the complete construction procedure, and we will dive deep in the following subsections

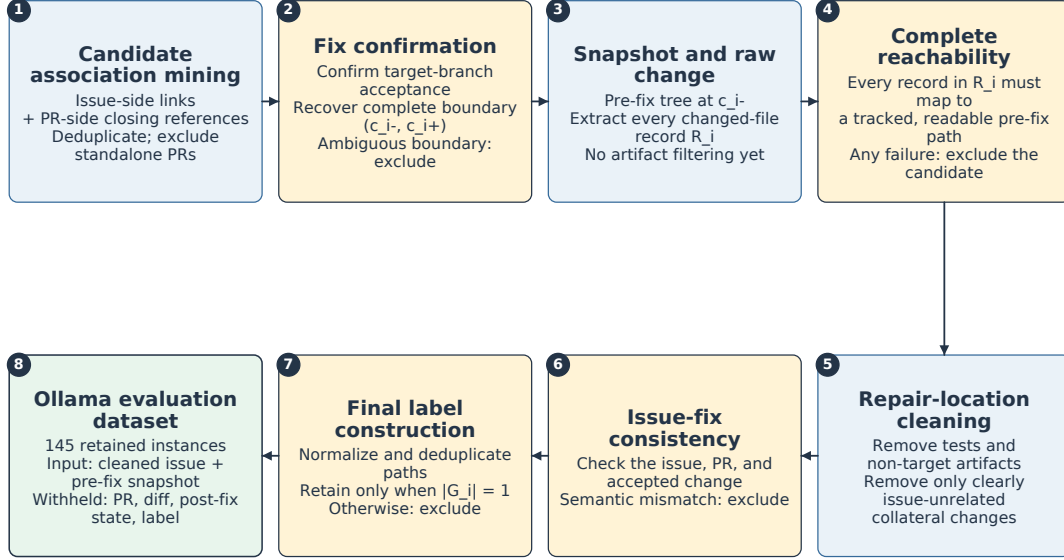


Figure 4.1.: Construction stages of the Ollama evaluation dataset.

4.1.1. Candidate Issue–PR Mining and Fix Confirmation

Figure 4.1 summarizes the eight-stage construction procedure. Stages 1–2 mine candidate issue–PR associations—from closed, bug-labelled issue timelines and from explicit closing references (fixes #N, closes #N, resolves #N) in merged PRs—and confirm that each proposed change was integrated. The bug label is only a mining cue; mislabelled reports are removed later by the consistency check (stage 6). Duplicates and standalone PRs are discarded because retained instances require an issue report as the localization query. For each candidate i , we identify the last pre-integration revision c_i^- and the corresponding post-integration revision c_i^+ (parent/merge or parent/commit); candidates with an ambiguous integration boundary are excluded.

4.1.2. Pre-Fix Snapshot Reconstruction and Raw Change Extraction

The tree at c_i^- is the *pre-fix snapshot* supplied to the localizer (stage 3); “pre-fix” only means the accepted change has not yet been applied, not that the failure was independently reproduced. We extract the complete file-level change $R_i = \text{ChangedFileRecords}(c_i^-, c_i^+)$ without any artifact or relevance filtering, mapping each record to its pre-fix path (old path for modifications, deletions, and unambiguous renames). New files, unresolved

renames/copies, gitlinks, and other records lacking an unambiguous pre-fix path are unreachable. Stage 4 requires every record in R_i to be reachable in the tracked, readable pre-fix file set $\mathcal{F}_{\text{read}}(c_i^-)$; any failure excludes the whole candidate, even if the offending record would later be filtered out anyway, so that no partially observable change is retained as if complete.

4.1.3. Repair-Location Cleaning and Final Label Construction

Candidates that pass reachability proceed through stages 5–7. A fixed policy first removes vendored dependencies, generated files, and build outputs; maintained implementation, test, script, and non-code artifacts remain eligible whenever the accepted change repairs the reported problem there (e.g., row 26 legitimately targets `server/routes_test.go`, since the execution-free protocol only prohibits running tests, not ranking maintained test source). For the later artifact-type analysis, *code/script* covers executable-language extensions (Go, C/C++, TypeScript, shell, PowerShell) and *non-code* covers documentation, build specifications, configuration, patches, templates, and model metadata. We then conservatively drop files that are clearly unrelated to the issue (e.g., bundled formatting or release bookkeeping), retaining a file when relevance is uncertain. A separate, instance-level consistency check excludes candidates whose PR only mentions the issue or whose accepted change addresses a different problem. Writing G_i for the unique, normalized repair-location paths remaining after artifact and relevance filtering,

$$\text{Retain}(i) \iff \text{Consistent}(i) \wedge |G_i| = 1,$$

so candidates with zero or multiple plausible repair locations are excluded. The resulting file is an operational *repair-location label*, not proof of a unique causal source.

4.1.4. Candidate Universe and Evaluation Inputs

The procedure yields the 145 retained instances (stage 8). For each, the candidate universe $U_i = \{p \in \mathcal{F}_{\text{read}}(c_i^-) \mid \text{EligibleArtifact}(p)\}$ satisfies $G_i \subseteq U_i$ and is independent of any approach’s chosen indexing strategy: predictions must be eligible repository-relative paths in U_i . Membership in U_i defines the files eligible for scoring and evaluation; it does not require a method’s returned ranking to enumerate every member of U_i . Construction may inspect the PR, fixing diff, and integration metadata to establish provenance; at evaluation time the localizer instead receives only the cleaned issue text and the pre-fix snapshot, with the PR description, fixing diff, integration revision, merge message, label, and post-fix contents withheld, and any history access bounded at c_i^- .

4.1.5. Final Dataset

The resulting Ollama evaluation dataset contains 145 issue–fix instances. Every retained instance has exactly one reachable and eligible repair-location label in its pre-fix snapshot. One label is a maintained Go test file; this does not introduce newly executed test evidence. Table 4.1 summarizes the final dataset. The dataset, including the cleaned issue reports, pre-fix revision identifiers, repair-location labels, and associated evaluation metadata, is publicly available at https://github.com/tianhaogutum/ollama_evaluation_dataset.

Table 4.1.: Composition of the Ollama evaluation dataset.

Property	Value
Instances	145
Final repair-location labels per instance	1
Ground Truth	one repository file

4.2. Study Setup

4.2.1. Studied Baselines

Table 4.2 summarizes the three evaluated approaches.

Table 4.2.: Overview of the evaluated localization approaches.

Method	Paradigm	Repository representation	Localization process
BugLocator	IRFL	Flat textual file representation plus similar historical reports	Deterministic one-shot positive-score ranking
BLUiR	IRFL	Separate report and code-structure fields	Deterministic one-shot positive-score ranking
SWE-agent	LLM-agent-based FL	Repository evidence inspected incrementally through shell tools	Iterative exploration followed by a ranked file list

BugLocator. The study uses a language-independent reimplementation of BugLocator rather than the original Java-specific artifact. It retains the two defining components of the approach: revised vector-space similarity (rVSM) between the issue report and

4. Evaluation of LLM-Agent-Based Fault Localization on Ollama Bugs

candidate files, and SimiScore evidence from previously fixed similar reports [ZZL12]. The two score vectors are independently min-max normalized and combined as

$$\text{Score}(f) = (1 - \alpha) N(\text{rVSM}(f)) + \alpha N(\text{SimiScore}(f)), \quad \alpha = 0.2. \quad (4.1)$$

The historical corpus is ordered chronologically: a prior issue-fix pair is eligible only when its fixing change was merged before the target issue was created. Identifiers are split at case and non-alphanumeric boundaries, converted to lowercase, filtered with English and programming-language stop-word sets, and stemmed with the Porter algorithm [Por80]. These adaptations make the representation applicable to Ollama’s multilingual files, but the implementation should be understood as a reimplementaion of the published scoring design, not as the authors’ original executable.

Figure 4.2 summarizes the two evidence channels and their fusion in the implementation evaluated in this study.

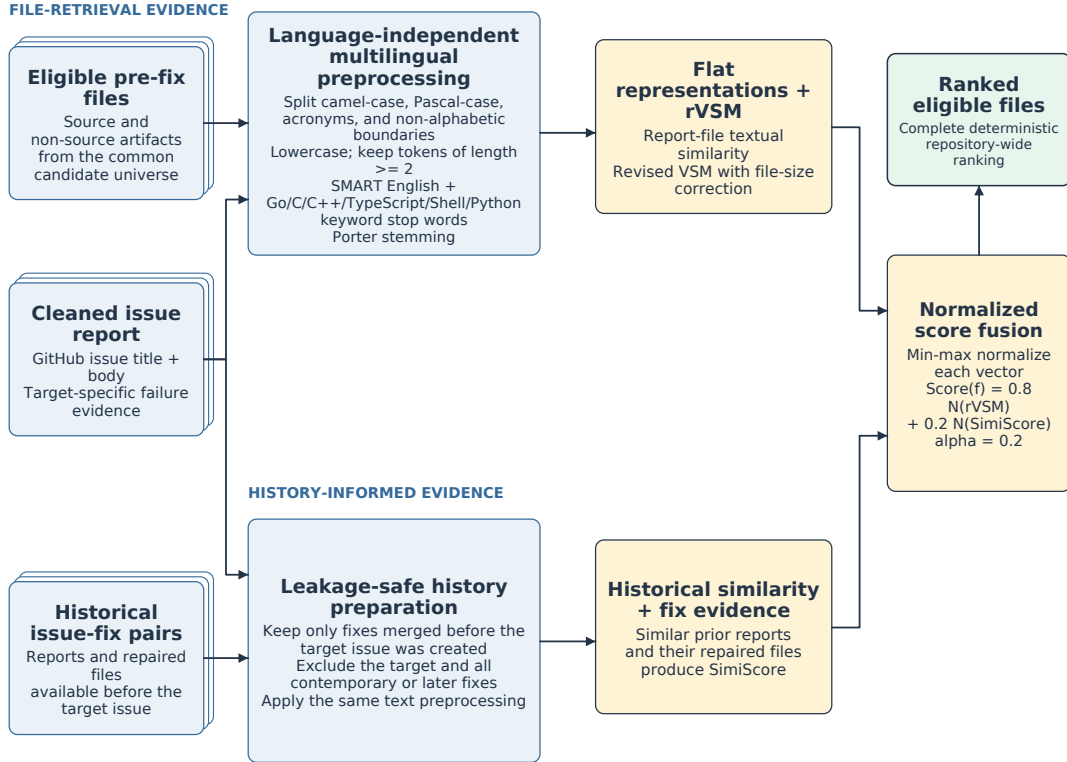


Figure 4.2.: Workflow of the multilingual BugLocator adaptation.

BLUiR. The study uses a multilingual adaptation of BLUiR. The original approach separates the issue summary and description and indexes Java class names, method names, variable names, and comments [Sah+13]. Here, the issue title and body provide the two query fields, while the Java-specific code fields are generalized to type names, function or method names, identifiers, and comments. Language-specific front ends extract these fields from Go, C-family, TypeScript/JavaScript, Python, shell, and PowerShell files. Eligible artifacts without comparable program structure use a plain-text fallback rather than being removed from the candidate universe.

BLUiR scores query-code field combinations with BM25 and sums the resulting scores for each file [RZ09]. The fixed parameters are $k_1 = 1.2$ and $b = 0.75$. Parser failures also fall back to the plain-text representation. This preserves representation and scoring coverage across the candidate universe but provides fewer structural fields for unsupported or non-code artifacts. In both IR implementations, every eligible file is scored, and the returned ranking contains all and only files with a non-zero final score, ordered by decreasing score. Zero-scored files remain in the candidate universe but are omitted from the output. As a reproducibility limitation, the current bundle does not retain per-file parser/fallback manifests for all 145 historical snapshots, so it does not report an aggregate parser-coverage percentage.

Figure 4.3 summarizes the multilingual front ends, shared structured representation, and plain-text fallback used by the evaluated implementation.

SWE-agent. The LLM-agent-based approach is a localization-only adaptation of SWE-agent [SWE26]. The framework and control loop are retained, while the YAML task configuration replaces patch generation with ranked file localization. Given an issue report and a checked-out pre-fix snapshot, the agent may list directories, search text, and read bounded file segments. It must finish by writing an ordered list of repository-relative paths; it is not asked to modify the program or propose a patch.

The evaluated configurations provide repository-navigation tools but no inference-engine software-layer map or history-informed localization guidance. Compilation, testing, model execution, network access, and modification of repository files are prohibited. The two archived deployments share the localization task, dataset, evaluator, and read-only execution setting. Exact task instructions, model identifiers, effective run settings, outputs, and trajectories are included in the replication package.

4.2.2. Backbone Model Selection

SWE-agent is instantiated with two backbone language models: GPT-5.2, a proprietary reasoning model, and DeepSeek V4 Pro, an open-weight model. The two backbones were selected to represent both a closed-source and an open-source model family,

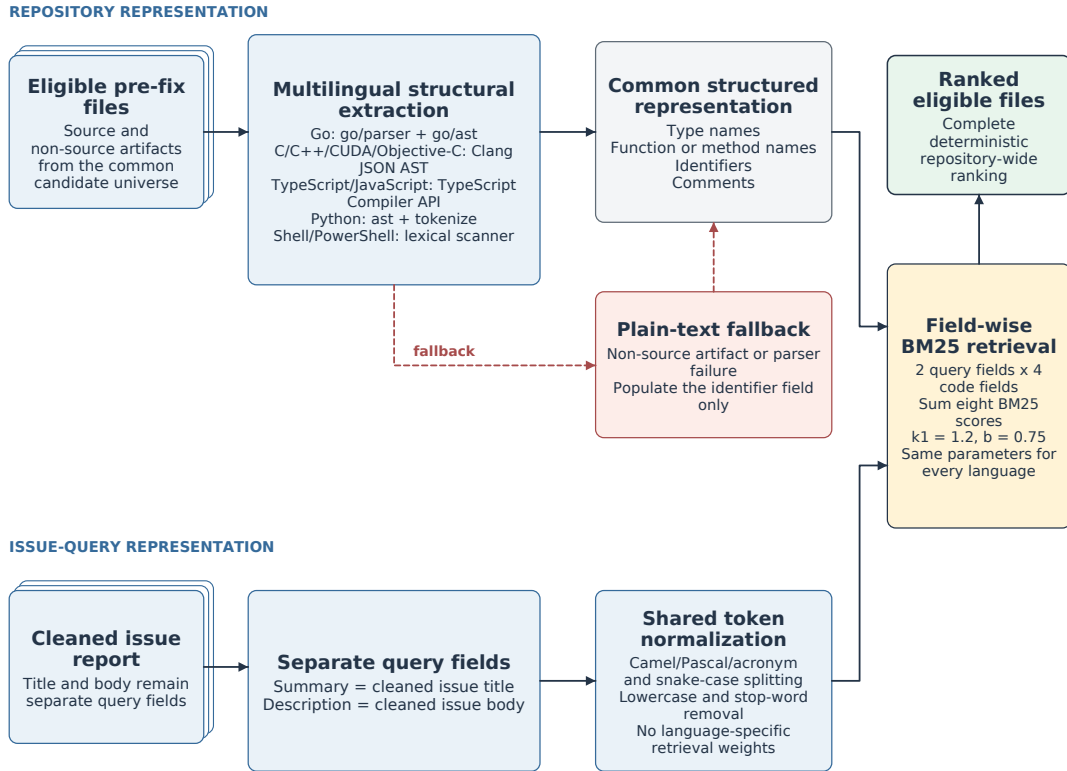


Figure 4.3.: Workflow of the multilingual BLUiR adaptation.

giving the evaluation some diversity in the underlying language model rather than relying on a single provider; the final pairing was also constrained by API cost. This pairing provides two representative agent configurations rather than an estimate of model-family performance, and it is not intended as a claim that either remains the strongest available model at the time of writing.

4.2.3. Implementation and Reproducibility

Table 4.3 records the settings that materially affect reproduction. No parameter is tuned on the evaluated instances. The retrieval approaches are run once because they are deterministic; the agent’s single-run design is retained because of its substantially higher cost, with complete trajectories archived for audit.

Local preprocessing and repository operations are performed on Ubuntu 22.04.5 with

Table 4.3.: Principal reproducibility settings.

Setting	BugLocator	BLUiR	SWE-agent
Implementation	Multilingual rVSM + SimiScore	Multilingual structured BM25	Localization-only SWE-agent
Fixed parameters	$\alpha = 0.2$	$k_1 = 1.2, b = 0.75$	–
Randomness	Deterministic	Deterministic	Stochastic
Runs per instance	One	One	One

a 48-core central processing unit (CPU) and 62 GiB of memory. The IR implementations use Python 3.10.12; the structured front ends additionally use the language toolchains documented in the replication package. Agent inference uses the OpenAI-compatible APIs identified above, while repository inspection runs locally in an isolated copy of the corresponding pre-fix snapshot. Because the two agent configurations use different endpoints, provider- and interface-specific effects remain part of the comparison.

4.3. Evaluation Metrics

Let $\mathcal{I}$ denote the set of evaluated instances, and let g_i denote the ground-truth repair file used by the evaluator for instance i . Let $R_i = (r_{i,1}, \dots, r_{i,5})$ denote the normalized returned ranking, possibly a strict subset of the candidate universe, truncated at five. The set of files in the first k positions is denoted by $R_{i,1:k}$, and let $g_i \sim r$ denote the normalized matching relation defined in Section 4.2: equality or a slash-delimited suffix match. Top- k accuracy is reported at $k \in \{1, 3, 5\}$, and reciprocal rank is truncated at five.

We cap the evaluation at $k = 5$ to operationalize a bounded, practitioner-oriented inspection scope rather than to characterize the full tail of each ranking. In a survey of 386 practitioners, Kochhar et al. found that 73.58% selected five program elements as their minimum success threshold, while more than 80% regarded a fault-localization session as successful only when the faulty element appeared within the top five positions [Koc+16]. Evaluating early ranks at $k \in \{1, 3, 5\}$ is also consistent with recent peer-reviewed fault-localization studies, including AutoFL, SoapFL, FlexFL, and DEVLoRe [KAY24; Qin+25b; Xu+25; Fen+26]. Accordingly, for the Ollama evaluation dataset, we report Top-1, Top-3, Top-5, and MRR@5 to measure whether the repair file is placed within a small candidate set and how early it appears within that set. The smaller incremental gain later observed from Top-3 to Top-5 for SWE-agent is treated as supporting evidence that its successful localizations are concentrated within this inspection scope, rather than as the reason for selecting the cutoff. Our conclusions

are therefore restricted to early-rank effectiveness within the top-five cutoff and do not characterize performance beyond rank five.

4.3.1. Effectiveness Metrics

Top- k accuracy measures the proportion of instances for which the repair file occurs in the first k positions:

$$\text{Top-}k = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{1}[\exists r \in R_{i,1:k} : g_i \sim r]. \quad (4.2)$$

Mean Reciprocal Rank at five (MRR@5) additionally rewards earlier placement. Let rank_i be the first rank whose prediction matches g_i under $\sim$, with $\text{rank}_i = \infty$ when it does not occur in the first five:

$$\text{MRR@5} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \begin{cases} 1/\text{rank}_i, & \text{rank}_i \leq 5, \\ 0, & \text{otherwise.} \end{cases} \quad (4.3)$$

4.3.2. Efficiency Metrics

The approaches have different execution models, so efficiency is characterized with approach-appropriate quantities. For BugLocator and BLUiR, end-to-end time includes candidate enumeration, preprocessing, temporary representation construction, scoring, and ranking. An implementation with a persistent index could amortize much of this work across reports, so the measured batch time is not a lower bound on production query latency.

For SWE-agent, the archived records provide counts of interaction steps and processed tokens, together with timestamp-based run durations, for all 145 instances. The GPT-5.2 provider reports monetary API cost directly. All 145 official DeepSeek trajectories record zero in the provider cost field; this field is therefore treated as unavailable, not as evidence of zero economic cost. Because the DeepSeek responses separately record cache-hit input, cache-miss input, and output tokens, we additionally compute a post-hoc list-price estimate. Across the 145 cases, these categories contain 429,715,968, 11,639,859, and 2,376,857 tokens, respectively. The prices per million tokens in force during the run were US\$0.003625 for cache-hit input, US\$0.435 for cache-miss input, and US\$0.87 for output. Applying these rates gives US\$8.6889 in aggregate, or US\$0.0599 per issue.¹ The observed quantities depend jointly on the model, prompt

¹DeepSeek’s historical price snapshot is available at <https://github.com/deepseek-ai/awesome-deepseek-agent/blob/862f00185e6e92f2705ef2642aa0621bbbec10d7/CONTRIBUTING.md>. The subsequently announced tariff took effect at 16:00 Coordinated Universal Time on 16 August 2026,

profile, provider load, accumulated context, and stopping behavior. The monetary-cost values are therefore experiment-specific quantities rather than intrinsic properties of LLM-agent-based localization.

4.4. Experimental Results

All three localization approaches are evaluated on the same 145 instances in the Ollama evaluation dataset using the common protocol described in Section 4.2. SWE-agent is instantiated once with GPT-5.2 and once with DeepSeek V4 Pro, resulting in four evaluated approach or deployment combinations overall. Table 4.4 reports their early-rank effectiveness. The highest value in each metric column is shown in bold. For every other value, a downward arrow reports its absolute gap from the column maximum, measured in percentage points for Top- k and in score units for MRR@5.

Table 4.4.: Localization effectiveness on the Ollama evaluation dataset.

Method	Top-1	Top-3	Top-5	MRR@5
BugLocator	23.45% ↓ 41.38	42.07% ↓ 37.93	48.97% ↓ 40.00	0.3320 ↓ 0.4068
BLUiR	21.38% ↓ 43.45	38.62% ↓ 41.38	52.41% ↓ 36.56	0.3187 ↓ 0.4200
SWE-agent (GPT-5.2)	51.03% ↓ 13.80	72.41% ↓ 7.59	75.17% ↓ 13.80	0.6138 ↓ 0.1249
SWE-agent (DeepSeek V4 Pro)	64.83%	80.00%	88.97%	0.7387

Neither IR approach places one quarter of the repair files first, although both recover approximately half by rank five. BugLocator is higher at Top-1 and Top-3 and obtains a higher MRR@5, whereas BLUiR is higher at Top-5. This cutoff-dependent reversal does not support declaring either representation uniformly superior. BugLocator’s historical signal can prioritize some recurring issues early, while BLUiR’s structured identifier matching can expand coverage deeper in the ranking. More broadly, the results do not establish that historical similarity is intrinsically more valuable than code structure, because the two implementations also differ in scoring, preprocessing, and language support.

Both SWE-agent configurations produce substantially more top-heavy rankings than the IR approaches. With GPT-5.2, the agent places the repair file first for 74 of 145 issues, within the first three for 105, and within the first five for 109. The corresponding counts for the DeepSeek V4 Pro configuration are 94, 116, and 129. It achieves the highest value in every effectiveness column, exceeding the evaluated GPT-5.2 configuration by 13.80 percentage points at Top-1, 7.59 points at Top-3, and 13.80 points at Top-5, together

after the final call in this run; see <https://api-docs.deepseek.com/news/news260813/>.

with a 0.1249 advantage in MRR@5. Because the prompt profiles and endpoints differ, these gaps compare complete configurations rather than isolating a backbone-model effect.

For both agent configurations, the largest gain from increasing the inspection scope occurs between Top-1 and Top-3. The gain from Top-3 to Top-5 is only 2.76 percentage points for GPT-5.2 and 8.97 points for DeepSeek V4 Pro, compared with gains of 21.38 and 15.17 points, respectively, from Top-1 to Top-3. Thus, successful localizations remain concentrated toward the top of the ranking. All four configurations use the same instances and evaluator, but each agent configuration represents one stochastic run and the two agent prompt profiles differ. Numerical differences should therefore not be interpreted as a stable or causal model ordering without repeated, prompt-matched evaluation.

Effectiveness represents only one dimension of the comparison. Table 4.5 reports the corresponding average interaction, token, and wall-clock statistics, together with the available API-cost measurement.

Table 4.5.: Average efficiency statistics of the evaluated localization approaches.

Method	Interaction steps	Recorded processed tokens	Wall-clock duration	API cost (USD)
BugLocator	–	–	3.88 s	–
BLUiR	–	–	107.39 s	–
SWE-agent (GPT-5.2)	20.81	248,731.85	71.26 s	\$0.1532
SWE-agent (DeepSeek V4 Pro)	52.57	3,060,225.41	277.13 s	\$0.0599

Note. A dash indicates that a metric is not applicable.

Table 4.5 reveals substantial differences in computational demand and in the reported monetary-cost values. BugLocator is the fastest approach, requiring only 3.88 seconds per instance on average. BLUiR requires 107.39 seconds because its evaluated cold-start implementation reconstructs structured representations and a temporary index for each repository snapshot. This duration therefore represents end-to-end preprocessing and localization rather than online query latency.

Among the agent configurations, GPT-5.2 uses fewer interaction steps and recorded processed tokens and completes an instance faster than DeepSeek V4 Pro. The GPT-5.2 archive records a mean API cost of US\$0.1532 per issue; the corresponding DeepSeek list-price estimate is US\$0.0599 per issue. Despite its higher token consumption, DeepSeek is cheaper per issue than GPT-5.2 under the providers’ different pricing schedules. BugLocator and BLUiR incur no external model-inference charges, but this does not imply zero local computational cost. Consequently, wall-clock duration, computational workload, reported monetary cost, and localization effectiveness represent distinct

dimensions of the overall trade-off.

Answer to RQ1. Yes: both IR-based and LLM-agent-based approaches produce useful file-level localization rankings for Ollama issues without new execution evidence, despite the repository’s scale, architectural coupling, and limited diagnostic evidence. BugLocator and BLUiR give imprecise but useful candidate reduction, ranking fewer than one quarter of repair files first but about half within the Top-5. Both SWE-agent configurations outperform the IR approaches at every cutoff; DeepSeek V4 Pro ranks 64.83% of repair files first and 88.97% within the Top-5. Partial clues in issue reports, combined with lexical, structural, or iterative-search evidence, let all approaches recover the repair file in a substantial share of cases, though none is reliable for every issue, and each agent result reflects a single stochastic run rather than a stability estimate.

4.5. Quantitative Analysis

4.5.1. Comparison with IR-based approaches

The evaluated IR implementations deterministically score every file in the eligible candidate universe and return all non-zero-scored files in decreasing score order, with little per-query interaction. They are effective when the issue report contains file names, API identifiers, model-format terms, or error strings that also occur in a repair file. BugLocator can additionally recover recurring bug families through historical report similarity, whereas BLUiR can emphasize terms appearing in function or type names rather than treating the entire file as undifferentiated text [ZZL12; Sah+13]. Quantitatively, BugLocator and BLUiR localize 34 and 31 of the 145 repair files at Top-1, respectively. By Top-5, these counts increase to 71 for BugLocator and 76 for BLUiR, corresponding to 48.97% and 52.41%, while their MRR@5 values remain 0.3320 and 0.3187. The two IR approaches do not follow a uniform ordering: BugLocator is equal or better at earlier cutoffs and achieves higher MRR@5, whereas BLUiR reaches a higher Top-5, indicating that BLUiR’s structured representation trades early-rank precision for broader recall rather than dominating BugLocator outright.

These mechanisms nevertheless depend on an observable lexical or semantic correspondence between the issue report and the file representation. In Ollama, an issue commonly describes an externally visible symptom, while the repair concerns scheduling, model metadata, native runtime behavior, hardware adaptation, or deployment logic, so vocabulary can diverge across software layers. Repeated generic terms such as *model*, *server*, or *graphics processing unit (GPU)* may also distribute retrieval scores across

many plausible files. Structured fields do not eliminate this problem when the relevant artifact has little program structure or when the report describes an observed effect rather than a code identifier.

The generic agent can respond to such uncertainty by inspecting the repository structure, reading candidate files, and revising its hypothesis, letting evidence accumulate across multiple search operations rather than through one-shot text matching, as in related tool-assisted or multi-round LLM-based localization approaches [KAY24; Qin+25b; Che+25]. Under the present protocol, which restricts the localizer to an issue report and a static pre-fix repository snapshot, SWE-agent with GPT-5.2 localizes 74, 105, and 109 repair files at Top-1, Top-3, and Top-5, respectively, while the DeepSeek V4 Pro configuration localizes 94, 116, and 129, substantially exceeding both IR approaches at every cutoff.

Case 107 (PR 9079) illustrates why this iterative exploration succeeds where lexical matching does not. The issue reports that the CLI progress display flickers during `pull` over a Secure Shell session, with no file name, function name, or error string tying the symptom to a location. BugLocator and BLUiR both fail to rank the repair file, `progress/progress.go`, within Top-5, since the report shares little surface vocabulary with the file beyond generic terms such as *display*. Both SWE-agent configurations instead rank `progress/progress.go` first: reasoning that a rendering artifact tied to terminal output implicates the progress-bar renderer lets the agent reach the correct file even though the report contains no direct lexical cue.

4.5.2. Uniqueness and Union

Aggregate Top-1 accuracy does not reveal whether the evaluated configurations localize the same instances or succeed on complementary subsets. To examine this distinction, Figure 4.4 presents the exclusive Top-1 intersections for two comparisons. An instance belongs to a set only when the corresponding configuration ranks its ground-truth repair file first. Consequently, every number in the figure denotes the number of instances in exactly that region rather than an inclusive intersection.

Figure 4.4(a) shows a pronounced asymmetry: GPT-5.2 alone accounts for 36 uniquely successful instances, versus three for BugLocator and one for BLUiR, with two more solved jointly by the IR approaches but not GPT-5.2. The three-way oracle union thus reaches 80 instances (55.17%), six beyond GPT-5.2 alone, though all three configurations agree on only 19 instances.

The two agent configurations overlap more strongly (Figure 4.4b): both rank the ground-truth file first in 68 instances, DeepSeek V4 Pro uniquely localizes 26 more, and GPT-5.2 uniquely localizes six, leaving 45 instances unsolved by either. Their union covers 100 instances (68.97%), a gain of 4.14 percentage points over DeepSeek V4 Pro

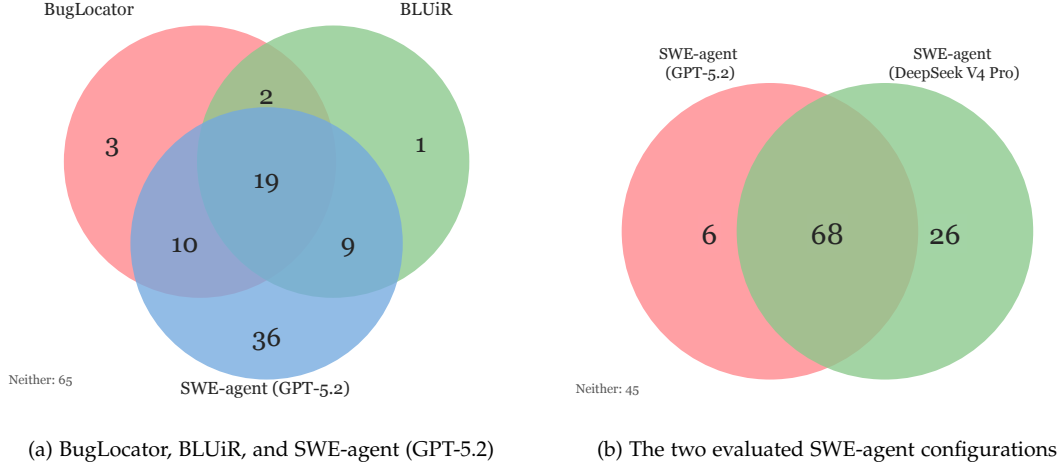


Figure 4.4.: Exclusive Top-1 intersections graph.

alone; since GPT-5.2’s six unique successes are not a subset of DeepSeek’s, and the two prompt profiles differ, this complementarity cannot be attributed to the backbone alone.

Combining all four configurations expands the union only slightly further, to 103 instances (71.03%), with the IR approaches contributing three cases that neither agent ranks first; 42 instances (28.97%) remain unsolved overall. This complete oracle exceeds the strongest standalone configuration by nine instances (6.21 points), but assumes perfect post-hoc selection and should be read as an upper bound on complementary coverage rather than a deployable ensemble.

4.5.3. Comparison on Different Artifact-Type

The candidate universe contains both source-code and non-source artifacts, but the evaluated designs represent them differently. BugLocator uses a flat textual representation that can index any readable artifact, although its effectiveness depends on lexical overlap between the issue report and the repair file. BLUiR extracts structural fields for supported programming languages but falls back to plain text for workflows, templates, documentation, build specifications, and other non-source artifacts.

To examine whether localization effectiveness varies with artifact type, we stratify the 145 retained instances according to the type of their single ground-truth repair location, using the extension-based rule defined in Section 4.1.3. The complete per-instance classification is included in the result artifact manifest.

Table 4.6 shows that every evaluated configuration performs worse when the repair location is a non-source artifact. At Top-5, the decrease is 24.16 percentage points for

Table 4.6.: Localization effectiveness by repair-location artifact type.

Method	Subset	Top-1	Top-3	Top-5	MRR@5
BugLocator	Source code	25.00%	47.32%	54.46%	0.3628
	Non-source	18.18% ↓ 6.82	24.24% ↓ 23.08	30.30% ↓ 24.16	0.2273 ↓ 0.1355
BLUiR	Source code	25.00%	43.75%	56.25%	0.3580
	Non-source	9.09% ↓ 15.91	21.21% ↓ 22.54	39.39% ↓ 16.86	0.1854 ↓ 0.1727
SWE-agent (GPT-5.2)	Source code	57.14%	79.46%	82.14%	0.6793
	Non-source	30.30% ↓ 26.84	48.48% ↓ 30.98	51.52% ↓ 30.62	0.3914 ↓ 0.2879
SWE-agent (DeepSeek V4 Pro)	Source code	67.86%	85.71%	92.86%	0.7769
	Non-source	54.55% ↓ 13.31	60.61% ↓ 25.10	75.76% ↓ 17.10	0.6091 ↓ 0.1678

BugLocator, 16.86 points for BLUiR, 30.62 points for SWE-agent with GPT-5.2, and 17.10 points with DeepSeek V4 Pro. The largest decreases generally occur at Top-3 and Top-5, indicating that non-source repair locations are substantially less likely to enter the practitioner-oriented inspection range. MRR@5 also decreases for every configuration, confirming weaker overall early-rank effectiveness on non-source artifacts.

A concrete instance illustrates this advantage. For PR 9773 (*Windows 11 Ollama 0.6.1 ROCm on gfx1151 is broken*), the accepted repair location is `CMakePresets.json`, a build-configuration artifact rather than program source. Neither BugLocator nor BLUiR returns this file within their ranked output for the instance. Both assign it a final score of zero, so it is omitted from their returned rankings of non-zero-scored files: the issue report’s GPU-architecture and driver-related vocabulary has no lexical or structural counterpart in the file’s terse JSON key–value content. SWE-agent with DeepSeek V4 Pro instead ranks the file first, consistent with its ability to iteratively inspect build and configuration artifacts alongside source files rather than relying on a single static representation computed in advance. This case exemplifies why non-source repair locations disproportionately benefit from iterative, artifact-agnostic inspection rather than one-shot lexical or structural retrieval.

4.5.4. Comparison on Computational Cost

The retrieval approaches incur no model-inference or API cost, but still require local computation: BugLocator averages 3.88 seconds per issue versus 107.39 for BLUiR, reflecting BLUiR’s added multilingual parsing rather than an inherent property of the two retrieval designs, since neither implementation maintains a persistent index across issues. The two SWE-agent configurations expose a different cost trade-off. GPT-5.2 averages 20.81 interaction steps, about 248,732 tokens, and 71.26 seconds per issue at US\$0.1532. DeepSeek V4 Pro explores more extensively (52.57 steps, about 3.06 million

tokens, 277.13 seconds); applying DeepSeek’s per-token pricing in force during the run to its recorded token categories gives US\$0.0599 per issue, cheaper than GPT-5.2 despite the higher token consumption. This comparison shows that agent configuration affects localization quality, interaction depth, latency, and token consumption, with DeepSeek achieving higher effectiveness while taking more steps, tokens, and time. This descriptive difference combines backbone, prompt, and provider effects and should not be read as a general ranking of the two language models; each configuration is also represented by only one stochastic run per instance.

Answer to RQ2. The evaluated approaches present a clear trade-off between efficiency and localization effectiveness. BugLocator and BLUiR are deterministic, require no model inference or API expenditure, and could further reduce their marginal per-issue latency through persistent repository indexing. SWE-agent produces substantially stronger early rankings by iteratively collecting and reassessing repository evidence, but requires fresh model inference for each issue and introduces greater latency, token consumption, external API use, and run-to-run variability.

4.6. Qualitative Analysis

Aggregate rank-based metrics reveal whether an accepted repair location enters the inspection scope, but they do not explain how an agent loses that location during repository exploration. We therefore manually inspected the failure trajectories of LLM-agent-based approaches.

4.6.1. Evidence Reachability Failures

The first failure occurs before the agent can compare causal explanations: the evidence needed to formulate the correct hypothesis never becomes semantically reachable. This problem is amplified by the scale and heterogeneity of inference-engine repositories. Table 1 of Liu et al. [Liu+26b] shows that the five engines in their sample range from approximately 122 thousand to 20.75 million lines of code and span C/C++, Python, and Compute Unified Device Architecture (CUDA) code. More importantly, Figures 1 and 2 distribute deployment logic across engine setup, model conversion, and inference/serving, with additional components for model loading, request processing, backend-specific operators, resource management, and hardware adaptation. Although these measurements do not characterize Ollama directly, they illustrate why a locally coherent search can cover only a small portion of the relevant evidence space.

Case 126 shows a concentration effect within a single subsystem. Long GPU-discovery logs from a GPU-less Windows machine led both agents through `discover/runner.go`, the scheduler, the runner lifecycle, and CUDA backend initialization. Neither agent opened `discover/cpu_windows.go`, where the accepted repair corrected the Windows CPU-discovery path. The agents did not search randomly: each assembled a plausible explanation from salient log evidence. The failure was that this evidence narrowed the search so strongly that a neighboring startup path never entered context.

This case does not establish that repository size alone causes localization failure. Rather, it shows a *coverage-allocation* problem that repository scale and architectural dispersion make more severe. A robust localizer should track which deployment phases and neighboring subsystems remain unexplored, instead of treating additional actions within the current subtree as evidence of broad repository coverage. The early exploration stage is therefore critical: once salient log evidence draws the agent into one subsystem, it should not prematurely rule out neighboring subtrees, since the initial subsystem chosen constrains almost all subsequent evidence gathering.

4.6.2. Causal-Chain Truncation

The second failure arises after relevant evidence has been found, but before the complete causal chain has been traversed. An inference request does not pass through one homogeneous implementation layer. Liu et al. [Liu+26b] model the serving path in Figure 2 as an engine initializer and model loader followed by a request parser, job manager, input processor, backend-specific operators, and output processor; resource managers, backend adapters, communicators, and device-specific runtimes cut across this path. Their Table 4 further shows that inference/serving contains all 28 identified root-cause types, while Table 5 maps the same visible symptom to functionality, environment, configuration, resource, and input/output causes. Thus, the component closest to an exception or malformed response is often the *failure-exposure layer*, rather than the layer that created the invalid state.

Case 86 is the clearest example. For a streaming response that lost its final token, GPT-5.2 followed the request through `server/routes.go`, `llm/server.go`, and `api/client.go`. It attributed the failure to Go-side scanning and stream ordering, but never crossed into the native response producer and returned a miss. DeepSeek continued into `llm/ext_server/server.cpp`, inspected the token and final-response path, and ranked the accepted repair location first. GPT-5.2 reconstructed a plausible prefix of the causal chain, but stopped at both a language and a process boundary. The desired behavior is therefore not simply to “go deeper” within one layer; it is to cross plausible language and process boundaries, identify where state first becomes invalid, and compare competing layer hypotheses before terminating the search.

4.6.3. Non-Source Artifact Underinspection

The third failure concerns artifact type rather than code depth. As shown in Table 4.6, Top-5 accuracy falls from 82.14% on code/script repair locations to 51.52% on non-code locations for the source-oriented GPT-5.2 configuration. For DeepSeek, the corresponding values are 92.86% and 75.76%, an observed code–non-code gap of 17.10 percentage points rather than GPT-5.2’s 30.62 points. The remaining gap reflects two observable mechanisms: some operational artifacts are never opened, whereas others are read but under-ranked relative to plausible implementation files. The smaller gap under the neutral profile is consistent with better artifact coverage, but the cross-configuration difference is not a controlled estimate of the prompt effect because the backbone, endpoint, and stochastic run also differ. The qualitative mechanism is deliberately broader than the binary stratum: it includes build/install scripts that the documented extension rule assigns to the Code/executable-script group, as well as Non-code artifacts.

Documentation cases illustrate this failure mode directly. In Case 103, GPT-5.2 confirmed that the requested API endpoint already existed and encountered an explicit link from the client code to `docs/api.md`, yet it continued inspecting implementation and test files without opening the document; DeepSeek inspected the document and ranked it fifth. This case shows that operational-artifact underinspection is a failure mode rather than an unavoidable property of document repairs.

Answer to RQ3. Yes: for the evaluated failure cases, manual trajectory coding attributes localization misses to three recurring, identifiable pitfalls rather than unexplainable model behavior: evidence non-reachability, where the repair file never enters the model’s effective context; causal-chain truncation, where the agent stops at a language, process, or component boundary before reaching the responsible layer; and non-source artifacts underinspection, where documentation, configuration, or build artifacts are never opened or are under-ranked.

4.7. Summary

This chapter evaluated IR-based and LLM-agent-based approaches for execution-free fault localization on Ollama issues. The results demonstrate the feasibility of these approaches, with agents achieving stronger early rankings (RQ1), though their iterative exploration incurs clear efficiency trade-offs in latency and token consumption (RQ2). Furthermore, manual trajectory analysis reveals that agent misses stem from a failure cascade of three identifiable reasoning breakdowns: evidence non-reachability, causal-

chain truncation, and non-source artifact underinspection (RQ3). To address these specific bottlenecks, the next chapter introduces CLUEFL, a domain-specific framework designed to further improve execution-free fault localization.

5. CLUEFL: Cross-Software-Layer Understanding of Evidence for Fault Localization

Chapter 4 showed that a generic repository agent is substantially more effective than the two IR-based localizers, but also exposed several systematic weaknesses. This chapter introduces CLUEFL, short for *Cross-Software-Layer Understanding of Evidence for Fault Localization*, as a domain-specific fault localization framework for LLM inference engines. CLUEFL combines three forms of guidance—project-specific knowledge, similar historical fixes, and a root-cause taxonomy—with a multi-step localization process that progressively narrows down the search space. It further provides a set of purpose-built tools for tracing upstream and downstream files along causal chains, retrieving code metadata to conserve context-window usage, and inspecting the source code of a specific function.

Beyond the bottlenecks identified in Chapter 4, the design of CLUEFL is also inspired by the observable practices of human developers during debugging. Empirical studies show that developers search for relevant code, follow incoming and outgoing dependencies, revisit previously examined locations, and progressively refine their understanding as new evidence is gathered [Ko+06; Law+13]. Developers also frequently pose reachability questions to trace how program behavior propagates across components [LM10]. More recent field evidence characterizes professional debugging as a structured and iterative diagnostic process in which developers continuously update their mental model, alternate between forward and backward tracing, and consult external resources to complement code-level evidence [LC26].

Rather than attempting to reproduce human cognition, CLUEFL operationalizes these observable practices. Project-specific knowledge anchors an initial architectural model; similar historical fixes serve as a proxy for prior debugging experience; the root-cause taxonomy encourages the consideration of multiple competing hypotheses; and the cross-layer navigation tools support targeted forward and backward evidence tracing before the candidate files are ranked. This human-inspired organization is also consistent with recent LLM-based fault localization systems such as AGENTFL, which structures localization into comprehension, navigation, and confirmation [Qin+24]. In

contrast, CLUEFL adapts this general principle to static, file-level localization in LLM inference engines, where faults may propagate across source code, build logic, and configuration artifacts.

Section 5.1 presents the framework, Section 5.2 defines the experimental setup, Section 5.3 reports the overall results, Section 5.4 analyzes those results, and Section 5.5 presents the case studies. The chapter summary answers RQ4.

5.1. Approach

5.1.1. Overview

CLUEFL is designed for the same pre-fix, repository-inspection setting as SWE-agent. The distinction lies in how the search process is orchestrated. Rather than treating the repository as an undifferentiated, flat collection of files, CLUEFL organizes localization into three prompt-defined phases: *hypothesis generation*, *evidence exploration*, and *relevance ranking*.

Figure 5.1 illustrates this process. To discourage premature commitment to a single symptom software layer, CLUEFL initiates *hypothesis generation* using an Ollama architecture map with eight software layers, a 28-category root-cause checklist, and similar issue–repair cases from the history. The prompt then asks for a ranking of all software layers with scores from 0 to 1, where 0 means not relevant to the given issue at all. During *evidence exploration*, CLUEFL guides the agent to extend its search beyond source code to non-source artifacts, such as build scripts and configuration files, which may also contain the actual repair locations. In addition to standard shell-based search, CLUEFL provides purpose-built repository-navigation tools for tracing upstream and downstream packages and files along causal chains, retrieving package-level metadata to conserve context-window usage, and inspecting the source code of individual functions on demand. These tools enable the agent to verify candidate hypotheses across software layers and progressively narrow the search space. Finally, during the *relevance ranking* stage, the agent assigns file-level confidence scores and returns all candidates with non-zero scores as a ranked list. The number of returned files is not fixed, thereby avoiding the forced inclusion of irrelevant files merely to satisfy a predefined quota. The evaluator then computes the Top- k metrics from the resulting list.

5.1.2. Hypothesis Generation

The first phase constructs a diverse, project-grounded hypothesis space before the agent begins fine-grained repository inspection. As illustrated in Figure 5.1, CLUEFL combines three complementary sources of guidance: a project-specific architectural

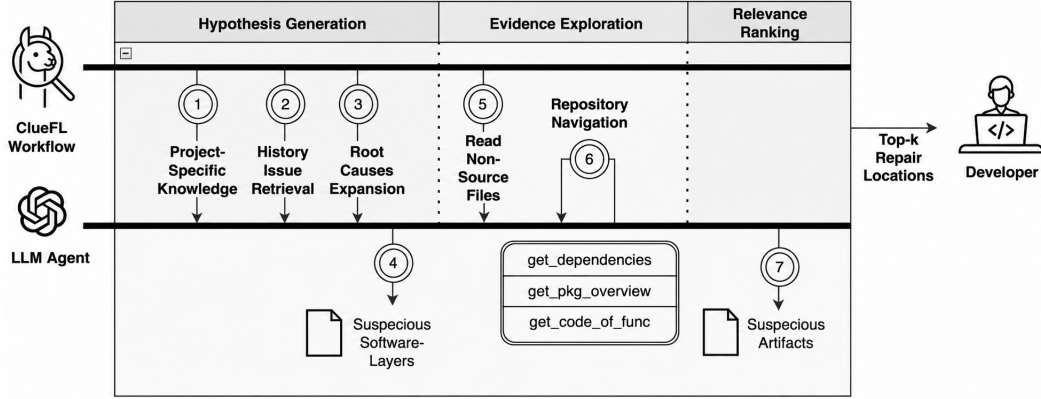


Figure 5.1.: Overview of CLUEFL.

map that indicates *where* a fault may reside, similar historical issue–fix pairs that reveal *where related faults were previously repaired*, and a root-cause taxonomy that broadens the agent’s reasoning about *why* the failure may occur. Together, these signals are used to assign a suspiciousness score to every software layer. Layers with non-zero scores are retained as the starting points for the subsequent evidence-exploration phase.

Project-Specific Knowledge

Modern LLM inference engines are repository-scale systems that integrate heterogeneous components spanning multiple programming languages and deployment phases. As observed in the failure analysis in Section 4.6, providing a generic agent with a flat repository view frequently leads to evidence-reachability failures and causal-chain truncation. Without a global architectural map, the agent struggles to traverse the semantic distance between an externally visible symptom and its underlying repair location. To mitigate this cross-software-layer distance, CLUEFL injects a project-specific software-layer taxonomy as an explicit prior for repository navigation.

We construct this prior through a reusable zero-shot procedure. First, we extract an inventory of package and interface nodes from the target repository. We then provide this inventory to Claude Opus 4.6 and ask it, without labeled examples or manually specified assignment rules, to group the nodes according to their functional responsibilities and positions in repository data and control flows. For the evaluated Ollama snapshot, the inventory contains 111 nodes. The zero-shot grouping assigns 69 nodes to eight named software layers and summarizes the remaining 42 nodes in three cross-cutting groups. The resulting taxonomy and cross-cutting groups are reported in Appendix A.2 rather than expanded here.

Although the resulting architectural map is specific to Ollama, its construction procedure is not. When applying CLUEFL to another inference engine, the same inventory-and-grouping procedure can be run on the new repository without requiring a manually labeled architectural dataset. The number, names, and boundaries of the generated layers may therefore differ across projects. In this sense, CLUEFL reuses the taxonomy-construction procedure rather than assuming that the Ollama-specific eight-layer taxonomy is universally applicable. The generated map is constructed once before fault localization and is then injected unchanged for all evaluated issues.

During the initial *hypothesis generation* phase, the agent consults this structured map to connect the observed symptom to plausible subsystems before beginning fine-grained repository inspection. For example, a CUDA out of memory report may prioritize the Infrastructure or Machine Learning (ML) Engine layer, whereas an OpenAI format error may prioritize HTTP API components. The associated execution paths further help the agent trace a symptom across layer boundaries rather than stopping at the component that merely exposes the failure. These macro-level coordinates focus subsequent shell-based inspection and reduce the risk of prematurely narrowing the search to a locally plausible but causally incomplete region.

Historical Issue Retrieval

To complement the architectural prior, CLUEFL leverages resolved issue–fix pairs to identify recurring bug patterns and fault-prone artifacts. Prior work has shown that bug histories, similar issue reports, and associated repair files provide useful localization evidence [Kim+07; ZZL12; WL14]. This design serves two purposes: to reuse project-specific developer knowledge and diagnostic experience, and to account for our observation that faults in LLM inference engines are concentrated in a small subset of error-prone files, consistent with prior findings for large software systems [OWB05].

CLUEFL represents issue text using term frequency–inverse document frequency (TF–IDF) and retrieves similar historical issues through cosine similarity. Similarity is computed from each issue’s title and truncated body. With a threshold of 0.05, all issues scoring above the threshold are returned, without imposing a fixed result count.

The agent receives the similarity score, issue title, and confirmed repair-file paths. Because the benchmark history provides neither fixing diffs nor root-cause explanations, these paths serve only as weak localization priors: they highlight potentially relevant or recurrently faulty artifacts but do not explain why the historical fixes worked. Historical context is therefore treated as a search heuristic rather than definitive evidence. In tightly coupled LLM inference engines, similar visible symptoms may arise from different root causes [Liu+26b]. Accordingly, the agent must inspect the current repository snapshot and assess each historical clue against current evidence. The final

candidate-file ranking remains an LLM judgment informed jointly by historical clues and evidence from the current repository.

Root Causes Expansion

While historical issues and architectural knowledge help the agent determine where to search, effective localization also requires plausible hypotheses about why the failure occurs. Our qualitative analysis in Section 4.6 showed that LLM-based agents often exhibit premature convergence and anchoring bias. After observing a symptom, they may fixate on the most common or immediately plausible root cause while overlooking alternative mechanisms, resulting in truncated causal reasoning.

To mitigate this diagnostic tunnel vision, CLUEFL introduces a root-cause expansion mechanism based on the empirical taxonomy of Liu et al. [Liu+26b]. The taxonomy identifies 28 root-cause categories specific to LLM inference engines, covering areas such as resource management, tensor operations, scheduling, hardware compatibility, and environment configuration. Rather than relying solely on the implicit priors acquired during LLM pre-training, CLUEFL injects these categories into the prompt as an explicit diagnostic checklist; the complete category list and diagnostic-pattern hints follow the taxonomy of Liu et al. [Liu+26b].

During *hypothesis generation*, the checklist captures potential mappings between failure symptoms and root-cause categories. For example, it reminds the agent that unexpected output may arise from distinct mechanisms involving formatting, configuration errors, numerical computation, or backend behavior. The checklist is not intended to make the agent mechanically enumerate all 28 categories. Instead, it provides a systematic diagnostic space for generating and comparing candidate hypotheses. By presenting multiple potential causal mechanisms, CLUEFL guides the agent to consider several plausible explanations before beginning fine-grained evidence exploration, reducing premature fixation on a single frequent root cause and establishing a more diverse hypothesis set for subsequent cross-software-layer evidence verification.

5.1.3. Evidence Exploration

The evidence-exploration phase transforms the high-level hypotheses generated in the preceding phase into concrete, file-level evidence. Before entering this phase, the agent assigns a suspiciousness score to every software layer based on the project-specific knowledge, retrieved historical issues, and expanded root-cause hypotheses. This layer score serves as a search gate rather than a final localization result: every layer with a non-zero suspiciousness score is retained for further exploration, whereas layers assigned a score of zero are excluded from the subsequent search. In this way, Step 5

produces a set of suspicious software layers that defines the scope of evidence collection without prematurely committing to a particular file.

The agent then reads all non-source files in the repository (Step 6). This is an exhaustive inspection rather than a symptom-dependent reminder. Such artifacts are relatively few and generally short in LLM inference-engine repositories, making complete inspection feasible within the available context window. The inspected artifacts include dependency manifests, build scripts, Dockerfiles, Makefiles, CMake files, configuration files, and other maintained non-source artifacts. This step prevents the agent from implicitly assuming that every fault must originate in runtime source code, and ensures that build-, dependency-, configuration-, and deployment-related repair locations remain visible throughout localization.

For source-code exploration, directly reading complete files can flood the context window with irrelevant implementation details and cause output truncation. Following the specialized-tool design principle of SWE-agent, CLUEFL therefore replaces unrestricted source dumping with domain- and language-aware repository-navigation utilities tailored to Ollama’s Go-dominated, Go/C/C++ architecture. The toolbox exposes three focused commands:

- `deps <package>` retrieves the recorded upstream and downstream dependencies of a package, allowing the agent to trace causal relationships across packages and software layers.
- `pkg_summary <package>` returns file summaries, imports, types, and function signatures without including the complete source files, enabling the agent to identify candidate files and functions at a lower context cost.
- `func <file> <function>` retrieves the implementation of a specified Go or C/C++ function, supporting targeted inspection of the code most directly related to the current hypothesis.

Using these utilities, the agent systematically explores every software layer with a non-zero suspiciousness score (Step 7). Dependency information is used to move along possible cross-layer causal chains, package summaries narrow the search from layers and packages to individual files, and function-level inspection provides implementation evidence for confirming or rejecting specific hypotheses. Generic commands such as `grep`, `find`, and bounded `sed` ranges remain available for lexical search and for artifacts not covered by the specialized tools.

The result of evidence exploration is not another set of layer- or package-level predictions. Instead, the agent produces a set of suspicious artifacts grounded in the collected repository evidence, with localization ultimately expressed at the *file level*.

Each retained file is associated with concrete evidence such as its role in a relevant dependency path, the presence of a suspicious function or configuration entry, or its consistency with the reported failure mechanism. These evidence-backed suspicious files are subsequently passed to the relevance-ranking phase.

5.1.4. Relevance Ranking

After hypothesis generation and evidence exploration, the final phase consolidates the collected evidence into a relevance ranking of suspicious artifacts. At this point, the high-level software-layer hypotheses have already been refined through dependency analysis, package inspection, function-level reading, and exhaustive examination of non-source artifacts. The ranking phase therefore operates at the *file level*: it does not rank software layers or packages, but the individual repository files that may require modification to resolve the reported issue.

For each candidate file, the agent assigns a relevance score in $[0, 1]$ based on the accumulated evidence and provides a brief justification for the score. The final output includes every file assigned a non-zero score and orders them by decreasing relevance; files omitted from the output are treated as having score zero. Consequently, the ranking is not restricted to a fixed minimum or maximum number of candidates. A strongly localized issue may produce only a small set of non-zero files, whereas a cross-layer or ambiguous issue may retain a broader candidate set.

This complete non-zero ranking supports a flexible downstream Top- k interface. Depending on the required inspection scope, the developer or evaluation procedure can select the k highest-scoring files as the predicted repair locations without rerunning the localization process. In our evaluation, the same ranked list is read at Top-1, Top-3, and Top-5, and is also used to compute MRR@5. Thus, k determines only how many results are returned from the ranking; it does not alter the evidence-collection or scoring process itself.

5.2. Experimental Setup

The experiment addresses RQ4 by comparing the generic baseline and CLUEFL under both DeepSeek V4 Pro and GPT-5.2. Within each comparison, the backbone model, issue reports, pre-fix repository snapshots, evaluation instances, and scorer are held fixed; only the localization configuration changes. Historical fixes supplied to CLUEFL are restricted to ancestors of each target snapshot, excluding the target fix and all later changes. The principal configurations are provided in Appendix A.1.

Both comparisons use the Top-1, Top-3, Top-5, and MRR@5 measures defined in Section 4.3. Table 5.1 summarizes the shared controls and intended differences for

the main comparison. For each backbone, we additionally evaluate four leave-one-component-out variants, removing in turn the software-layer knowledge, historical issue retrieval, root-cause taxonomy, or navigation toolbox while retaining the other components. The API does not expose a controllable random seed, so each configuration contributes one archived run per instance.

Table 5.1.: Controlled settings for the RQ4 comparison.

Setting	Generic baseline	CLUEFL
Backbone model	DeepSeek V4 Pro or GPT-5.2	Same
Evaluation instances	Ollama Evaluation Dataset	Same
Repository snapshot	Target pre-fix revision	Target pre-fix revision
Artifact policy	Generic, unguided	Guided exploration
Historical Issues	None	Yes
Navigation interface	Generic shell tools	Plus designed toolbox
Required output	Ranked candidate artifacts	Same
Evaluation cutoffs	Top-1/3/5	Top-1/3/5

5.3. Experimental Results

Table 5.2 reports the matched comparisons for DeepSeek V4 Pro and GPT-5.2. Counts in parentheses show the number of issues whose repair file appears within each cutoff.

Table 5.2.: CLUEFL Comparison with SWE-agent across different LLMs

Method	Top-1	Top-3	Top-5	MRR@5
GPT-5.2				
SWE-agent baseline	51.03% (74)	72.41% (105)	75.17% (109)	0.6138
CLUEFL	58.62% (85) $\uparrow 7.59$	75.86% (110) $\uparrow 3.45$	81.38% (118) $\uparrow 6.21$	0.6767 $\uparrow 0.0629$
DeepSeek V4 Pro				
SWE-agent baseline	64.83% (94)	80.00% (116)	88.97% (129)	0.7387
CLUEFL	67.59% (98) $\uparrow 2.76$	82.76% (120) $\uparrow 2.76$	88.97% (129) $\rightarrow 0.00$	0.7583 $\uparrow 0.0195$

Across both backbones, CLUEFL improves file-level localization. With DeepSeek V4 Pro, the gain is concentrated in earlier placement within the ranked list, increasing Top-1 accuracy by 2.76 percentage points (from 64.83% to 67.59%) and MRR@5 by 0.0195, while Top-5 coverage remains unchanged at 88.97%. With GPT-5.2, the improvement is larger and extends across all reported cutoffs, boosting Top-1 by 7.59 percentage points (51.03% to 58.62%) and Top-5 by 6.21 points (75.17% to 81.38%).

The magnitude of improvement varies with the observed generic-agent baseline score. The DeepSeek V4 Pro configuration begins with a Top-5 score of 88.97%, leaving less room for additional guidance. The GPT-5.2 configuration has the lower observed baseline score (a Top-5 of 75.17%) and shows a larger improvement with the architectural, historical, and root-cause guidance. These results suggest that CLUEFL may provide greater gains when the corresponding unguided baseline leaves more room for improvement.

5.4. Quantitative Analysis

5.4.1. Uniqueness and Union

Figure 5.2 decomposes the Top-1 totals. The GPT-5.2 configurations share 70 successes, with 15 unique to CLUEFL and four to the baseline; the corresponding DeepSeek V4 Pro counts are 88, ten, and six. Thus, the gains do not represent strict superset coverage.

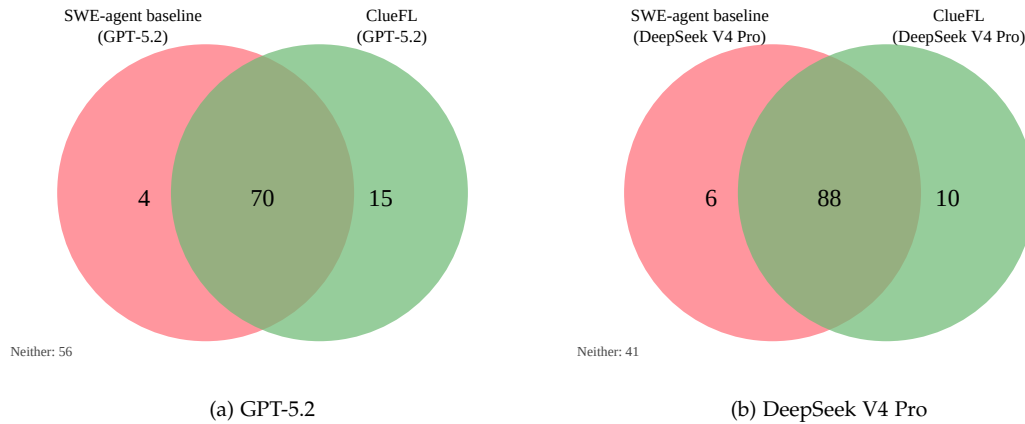


Figure 5.2.: Exclusive Top-1 success intersections between SWE-agent and CLUEFL.

Taken as oracle unions, the two GPT-5.2 configurations cover 89 instances (61.38%), four more than CLUEFL alone, while the DeepSeek V4 Pro configurations cover 104 instances (71.72%), six more than CLUEFL alone. These unions assume perfect post-hoc selection between two stochastic runs and therefore indicate complementary coverage rather than the expected performance of a deployable ensemble.

5.4.2. Comparison on Different Artifact Type

The quantitative analysis in Chapter 4 (Section 4.5) shows that non-source repair artifacts are generally harder to localize than source code. Because CLUEFL examines non-

source artifacts during evidence exploration (Section 5.1.3), we stratify both backbone comparisons over the same extension-based split: 112 source-code instances and 33 non-source instances. Table 5.3 reports Top-1, Top-3, Top-5, and MRR@5 for these configurations across both GPT-5.2 and DeepSeek V4 Pro.

Table 5.3.: Matched localization effectiveness by repair-location artifact type across LLM backbones.

Subset	Method	Top-1	Top-3	Top-5	MRR@5
GPT-5.2					
Source code	SWE-agent	57.14%	79.46%	82.14%	0.6793
	CLUEFL	64.29% $\uparrow 7.15$	80.36% $\uparrow 0.90$	87.50% $\uparrow 5.36$	0.7347 $\uparrow 0.0554$
Non-source	SWE-agent	30.30%	48.48%	51.52%	0.3914
	CLUEFL	39.39% $\uparrow 9.09$	60.61% $\uparrow 12.13$	60.61% $\uparrow 9.09$	0.4798 $\uparrow 0.0884$
DeepSeek V4 Pro					
Source code	SWE-agent	67.86%	85.71%	92.86%	0.7769
	CLUEFL	72.32% $\uparrow 4.46$	88.39% $\uparrow 2.68$	93.75% $\uparrow 0.89$	0.8107 $\uparrow 0.0338$
Non-source	SWE-agent	54.55%	60.61%	75.76%	0.6091
	CLUEFL	51.52% $\downarrow 3.03$	63.64% $\uparrow 3.03$	72.73% $\downarrow 3.03$	0.5803 $\downarrow 0.0288$

For GPT-5.2, CLUEFL gains at every metric on both subsets: Top-5 rises by 5.36 points on source code (92 to 98) and by 9.09 points on non-source artifacts (17 to 20), with matching gains on Top-1, Top-3, and MRR@5. Since the non-source subset contains only 33 cases, each case changes an accuracy value by 3.03 points. Non-source artifacts remain harder overall, but the source–non-source Top-5 gap narrows from 30.62 points under the baseline to 26.89 points under CLUEFL, consistent with its explicit encouragement to search beyond source code during evidence exploration.

On DeepSeek V4 Pro, the split is not uniformly positive. CLUEFL gains at every metric on source code (Top-5 up 0.89 points, 104 to 105), but on the 33-case non-source subset the result is mixed: Top-3 improves by 3.03 points while Top-1, Top-5, and MRR@5 each fall by one case (3.03 points, or 0.0288 for MRR@5). Unlike GPT-5.2, where CLUEFL narrows the source–non-source Top-5 gap, here the gap widens from 17.10 to 21.02 points: DeepSeek’s already strong non-source baseline (75.76% Top-5) leaves little room for a matching improvement, and the observed difference is within the range attributable to a single-case change. This is consistent with the chapter’s broader finding that CLUEFL’s benefit is not uniform across artifact types and backbones.

5.4.3. Component Ablation

To examine whether the improvement of the full framework depends primarily on a single design element, we remove each component in turn while retaining the other

three components and the same evaluation protocol. The four variants exclude software-layer knowledge, historical issue retrieval, the root-cause taxonomy, or the navigation toolbox, respectively. Table 5.4 reports the resulting sensitivity of both backbone configurations. Because a component is removed from the complete system rather than evaluated in isolation, the observed decrease reflects both its direct role and its interaction with the components that remain.

Table 5.4.: Leave-one-component-out ablation on the 145 evaluation instances.

Configuration	Top-1	Top-3	Top-5	MRR@5
GPT-5.2				
Full CLUEFL	58.62%	75.86%	81.38%	0.6767
– Layer knowledge	54.48% ↓ 4.14	73.79% ↓ 2.07	77.93% ↓ 3.45	0.6445 ↓ 0.0322
– Historical retrieval	55.86% ↓ 2.76	74.48% ↓ 1.38	79.31% ↓ 2.07	0.6531 ↓ 0.0236
– Root-cause taxonomy	56.55% ↓ 2.07	75.17% ↓ 0.69	80.00% ↓ 1.38	0.6605 ↓ 0.0162
– Navigation toolbox	55.17% ↓ 3.45	73.79% ↓ 2.07	78.62% ↓ 2.76	0.6490 ↓ 0.0277
DeepSeek V4 Pro				
Full CLUEFL	67.59%	82.76%	88.97%	0.7583
– Layer knowledge	65.52% ↓ 2.07	80.69% ↓ 2.07	88.28% ↓ 0.69	0.7420 ↓ 0.0163
– Historical retrieval	66.21% ↓ 1.38	81.38% ↓ 1.38	88.97% → 0.00	0.7470 ↓ 0.0113
– Root-cause taxonomy	66.90% ↓ 0.69	82.07% ↓ 0.69	88.97% → 0.00	0.7520 ↓ 0.0063
– Navigation toolbox	66.21% ↓ 1.38	81.38% ↓ 1.38	88.28% ↓ 0.69	0.7460 ↓ 0.0123

For GPT-5.2, removing any component reduces all four effectiveness measures. Software-layer knowledge has the largest observed effect: without it, Top-1 falls by 4.14 percentage points, Top-5 by 3.45 points, and MRR@5 by 0.0322. Removing the navigation toolbox produces the next-largest decrease in Top-1, Top-5, and MRR@5 (3.45 points, 2.76 points, and 0.0277, respectively), while its Top-3 decrease ties that caused by removing layer knowledge. Historical retrieval has a smaller but still consistent effect, reducing Top-1 by 2.76 points and MRR@5 by 0.0236. The root-cause taxonomy is the least sensitive removal for this backbone, but its absence still lowers Top-1 by 2.07 points, Top-5 by 1.38 points, and MRR@5 by 0.0162. The agreement across the four metrics indicates that the GPT-5.2 improvement is not produced solely by one source of guidance.

DeepSeek V4 Pro exhibits the same direction of change at Top-1, Top-3, and MRR@5, but its Top-5 coverage is less sensitive. Removing software-layer knowledge again produces the largest degradation, lowering Top-1 and Top-3 by 2.07 points each and MRR@5 by 0.0163. Historical retrieval and the navigation toolbox yield equal 1.38-point reductions at Top-1 and Top-3; navigation has a slightly larger MRR@5 decrease (0.0123 versus 0.0113) and is the only one of the two to reduce Top-5. Removing the root-cause taxonomy produces the smallest change, with decreases of 0.69 points at Top-1 and

Top-3 and 0.0063 in MRR@5. Top-5 remains unchanged when either historical retrieval or the root-cause taxonomy is removed. Together with the lower MRR@5 values, this aggregate pattern is consistent with weaker ordering inside the top five even when overall Top-5 coverage is preserved.

Across both backbones, the MRR@5 sensitivity follows the same ordering: software-layer knowledge, navigation toolbox, historical retrieval, and root-cause taxonomy. Moreover, every ablated configuration remains above its backbone-matched SWE-agent baseline at Top-1, Top-3, and MRR@5, although the DeepSeek variants without layer knowledge or navigation are each one Top-5 case below that baseline. This comparison suggests that no individual component alone accounts for the full framework’s improvement; the remaining components continue to provide useful guidance after any single removal. Nevertheless, leave-one-component-out differences are not additive estimates of independent component effects, and each stochastic configuration is represented by only one run per instance. The results therefore provide descriptive evidence about component sensitivity rather than a stable causal ranking of their marginal contributions.

5.5. Qualitative Analysis

5.5.1. Impact of Project-Specific Layer Knowledge

An inference-engine repository does not present its causes and symptoms within one homogeneous layer: build configuration, runtime scheduling, and native inference code are separated concerns that a purely code-centric search does not distinguish. CLUEFL makes this separation explicit by supplying an architecture map of software layers and a root-cause checklist during hypothesis generation (Section 5.1.2), so that the agent can consider a build-system explanation as a distinct hypothesis rather than as an extension of the runtime code it has already read.

PR 9425 illustrates this mechanism directly. The reported symptom is a throughput regression on a newer GPU architecture, and both agents build the same high-level hypothesis: a backend-selection or build-configuration change altered the CUDA execution path. The SWE-agent baseline pursues this hypothesis entirely within the runtime code, ranking `llama/llama.go`, `runner/llamarunner/runner.go`, and `server/sched.go` as the most suspicious files; the ground-truth repair location, the top-level `CMakeLists.txt`, never appears among its eleven ranked candidates. CLUEFL reaches a different top candidate: guided by the architecture map, it treats the build system as a distinct software layer from the runtime and inspects it directly. Its evidence notes that the runner log reports `GGML_CUDA_GRAPHs` enabled and a non-default `GGML_CUDA_PEER_MAX_BATCH_SIZE`, both of which are compile-time toggles defined in `CMakeLists.txt` rather than in any

runtime file; on this basis, CLUEFL assigns `CMakeLists.txt` the highest confidence score and ranks it first, while retaining the runtime-code hypotheses, such as buffer-type selection in `llama-model.cpp` and offload estimation in `llm/memory.go`, as secondary candidates rather than discarding them outright.

This case suggests that explicitly separating the build-system layer from runtime code, rather than searching source code alone, is what allows CLUEFL to consider a repair location that a code-centric search strategy does not reach. The mechanism is architectural framing rather than additional search effort: both agents reasoned about the same log evidence, but only the agent supplied with layer knowledge treated the build system as a candidate layer in its own right.

5.5.2. Impact of Historical Issue Retrieval

Beyond architectural layer knowledge, CLUEFL retrieves resolved issue–fix pairs whose text is similar to the current report (Section 5.1.2). Because faults in a given project tend to recur in a small set of error-prone files, a sufficiently similar historical issue can point directly at the current repair location, even when the current symptom text alone supports several plausible root causes.

PR 13445 shows this mechanism in a case where the surface symptom is ambiguous. The report describes Gemma 3 producing gibberish output, including literal special-token text, after an upstream version bump. The SWE-agent baseline commits to a tokenization hypothesis: it ranks `model/models/gemma3/model.go`, `convert/tokenizer_spm.go`, and the Gemma 3 prompt templates highest, reasoning that end-of-turn token handling regressed. The ground-truth repair location, `model/models/gemma3/model_text.go`, does not appear among its top ten candidates. CLUEFL instead retrieves a near-duplicate historical issue, titled almost identically to the current report, whose confirmed fix touched exactly `model/models/gemma3/model_text.go`; a second, related historical issue returns the same file. On this basis, CLUEFL ranks `model/models/gemma3/model_text.go` first, attributing the regression to Gemma 3’s sliding-window and cached-attention logic rather than to tokenization.

This case suggests that historical issue retrieval is most useful precisely when the current symptom is compatible with more than one plausible subsystem: rather than requiring the agent to disambiguate root causes from the current evidence alone, a sufficiently similar past fix narrows the search directly to a recurrently faulty file. The retrieved fix remained a prior rather than a verified explanation, but the file it pointed to matched the accepted repair location that a tokenization-centric reading of the symptom did not.

5.5.3. Impact of Root-Cause Expansion

The two mechanisms above help the agent decide *where* to look; the root-cause expansion mechanism described in Section 5.1.2 instead broadens *why* a symptom might be occurring, guarding against the premature anchoring on a single plausible explanation observed in Section 4.6.

PR 6905 illustrates this effect. The report describes the Windows graphical user interface (GUI) build of Ollama running noticeably slower than the same version launched from the command line. Both agents correctly infer that some Windows-only code path distinguishes GUI-launched from CLI-launched execution, but they commit to different explanations. The SWE-agent baseline anchors on a scheduling hypothesis: it ranks `app/tray/wintray/w32api.go` highest, reasoning that the tray icon’s `runtime.LockOSThread()` call and its Windows message-pump event loop could starve other goroutines. This is a plausible mechanism, but it leads the agent to focus on tray- and lifecycle-related files; the ground-truth file, `llm/llm_windows.go`, is ranked only sixth. Prompted with the root-cause checklist, CLUEFL instead considers an environment/runtime-mismatch category (RD.1/RD.2) alongside the scheduling explanation, hypothesizing that the GUI process inherits a different Windows process environment than the CLI, changing how the external runner subprocess is spawned and which CUDA libraries it loads. This hypothesis points directly at `llm/llm_windows.go`, which sets the process creation flags for the spawned runner, and CLUEFL ranks it first.

This case shows the checklist’s role concretely: rather than replacing the agent’s own reasoning, it adds an alternative causal category to the hypothesis set early enough that the agent verifies it against the evidence before settling on a single explanation.

5.5.4. Impact of Repository Navigation

The preceding case studies concern which layer or file to suspect; the repository-navigation toolbox instead determines how precisely that suspicion can be verified once a candidate is identified. In place of unrestricted source dumping, CLUEFL exposes `deps`, `pkg_summary`, and `func` so that the agent can trace cross-package dependencies, inspect package-level signatures, and read individual function bodies without flooding the context window (Section 5.1.3).

PR 5121 shows these tools converging on a precise, function-level root cause. The report describes DeepSeek-V2-Lite failing to load with a CUDA out-of-memory error immediately followed by a GGML shape assertion. Both agents correctly identify the general memory-estimation subsystem (`llm/memory.go`, `llm/server.go`) as suspicious, but the SWE-agent baseline can only gesture at it generically: it ranks `llm/ggml.go` far down its candidate list, reasoning only that the file provides “metadata used

by `EstimateGPULayers`” that could plausibly be mishandled. CLUEFL instead uses `deps llm/` and `deps llama/` to confirm the dependency boundary between the scheduler and the runtime layer, `pkg_summary` to shortlist `llm/memory.go`, `llm/server.go`, and `llm/ggml.go`, and then reads the implementations of `EstimateGPULayers` and `GraphSize` directly with `func`. This reveals that `GraphSize` has no case for the `deepseek2` architecture and silently returns zero, forcing `EstimateGPULayers` into a crude fallback that misestimates video random-access memory and triggers the reported allocation failure. Grounded in this function-level evidence, CLUEFL ranks `llm/ggml.go` second with an explicit, verifiable mechanism, rather than a generic file-level guess.

This case illustrates the toolbox’s intended effect: it lets the agent turn a plausible file-level suspicion into a specific, checkable claim about a single function, without paying the context cost of reading each candidate file in full.

5.6. Summary

This chapter presented CLUEFL, which combines software-layer knowledge, history-informed guidance, root-cause expansion, and a structured workflow for hypothesis generation, evidence exploration, and file-level ranking. The evaluation compares the complete approach with the generic baseline and leave-one-component-out variants under two backbone models while preserving the same input for fault localization. Every component removal lowers MRR@5 for both backbones, with the largest observed decrease following removal of software-layer knowledge.

Answer to RQ4. Yes: incorporating domain-specific knowledge and a structured localization workflow improves fault localization across both evaluated backbone configurations. The larger observed gain occurs for the configuration with the lower generic-agent baseline score, suggesting that the guidance may provide greater benefits when the unguided baseline leaves more room for improvement.

6. Discussion and Conclusion

6.1. Threats to Validity

This section discusses the construct, internal, and external validity threats to this study.

Construct validity. The Ollama evaluation dataset treats files modified by the developer patch as ground-truth fault locations. Although this is a common operationalization of file-level fault localization, other valid fixes may modify a different file, and this remains a challenge for fault localization research.

Internal validity. The main internal threat is potential contamination of pre-training data. The Ollama evaluation dataset is constructed from the public GitHub repository of Ollama, and some issue reports, pull requests, and fixing changes may therefore have appeared in the pretraining data of the evaluated language models. For example, GPT-5.2 has a documented knowledge cutoff of 31 August 2025,¹ which overlaps with part of the cases in the Ollama evaluation dataset. For DeepSeek V4 Pro, the complete training corpus and an equally precise knowledge cutoff are not publicly disclosed.² Consequently, pretraining-data contamination cannot be completely excluded for either backbone.

Another internal threat concerns agent nondeterminism. The provider APIs do not expose a controllable random seed, and each agent configuration is represented by one archived run per issue. The reported effectiveness and efficiency differences are therefore descriptive observations for the evaluated runs rather than estimates of run-to-run variance. Repeated trials would be required to quantify the stability of file rankings, interaction depth, token consumption, latency, and cost.

External validity. The empirical evaluation is conducted exclusively on issue-fix pairs collected from the Ollama repository. While Ollama supports a wide range of programming languages, artifact types, and software subsystems, its architecture and development trajectory focus on specific design goals that may differ from those of other LLM inference engines, such as vLLM, llama.cpp, or TensorRT-LLM. Consequently, these findings serve as a preliminary insight, and more extensive cross-project evaluations are needed to confirm their broader applicability.

¹<https://developers.openai.com/api/docs/models/gpt-5.2>

²<https://api-docs.deepseek.com/news/news260424/>

6.2. Future Research

Based on the limitations and findings of this thesis, we propose the following directions for future research:

1. **Root-Cause-to-Fix-Strategy Knowledge for Fault Localization:** Liu et al. [Liu+26b] map their root-cause taxonomy to common fix strategies observed in LLM inference engine repositories. A predicted root cause may narrow the set of plausible fix strategies, while those strategies may in turn indicate the software layers, artifact types, or files most likely to require modification. It is therefore worth examining whether incorporating this root-cause-to-fix-strategy mapping as external diagnostic knowledge can indirectly assist file-level fault localization and improve the candidate ranking produced by CLUEFL.
2. **Cross-Engine and Cross-Backbone Evaluation:** The current evaluation studies a single LLM inference engine, Ollama, with GPT-5.2 and DeepSeek V4 Pro as two representative backbone models. Future work should construct comparable issue–fix datasets for additional inference engines, such as vLLM, llama.cpp, and TensorRT-LLM, and evaluate CLUEFL with a broader range of proprietary and open-weight backbones. This would test whether the observed improvements generalize across repository architectures and model configurations with different reasoning capabilities, tool-use behavior, context capacities, and computational costs.
3. **Repeated-Run Robustness and Efficiency Evaluation:** The current study reports interaction steps, processed tokens, wall-clock duration, and available or estimated API cost, but each agent configuration contributes only one stochastic run per issue. Future work should repeat each matched configuration and report the variation in Top- k accuracy and MRR@5 together with distributions of token usage, tool calls, latency, and monetary cost. Such an evaluation would clarify both the stability of the rankings and the tail behavior of the effectiveness–efficiency trade-off.
4. **User Study of CLUEFL:** Although a practitioner survey suggests that inspecting at most five suspicious program elements constitutes an acceptable fault-localization success criterion for many developers [Koc+16], and a prior user study shows that fault-localization tools can significantly reduce professional developers’ debugging time [Xia+16], there is currently no direct evidence that CLUEFL can reduce repair time or improve repair accuracy for developers maintaining the Ollama repository. Therefore, future user studies should invite Ollama maintainers or

contributors to perform real-world software maintenance tasks and evaluate the practical impact of CLUEFL on repair efficiency and correctness.

6.3. Summary

In this thesis, we evaluated various file-level fault localization approaches for LLM inference engines by constructing a dataset of existing Ollama issue-fix pairs. We then developed the CLUEFL framework and demonstrated that domain-specific guidance and a well-designed workflow, as implemented in CLUEFL, improve an agent’s ability to prioritize repair locations for Ollama issues. By moving beyond flat repository exploration toward cross-software-layer understanding, this work provides a foundation for diagnostic tools that support fault localization and maintenance in large-scale LLM infrastructure.

A. Supplementary Materials

A.1. Agent Prompts

This section documents the prompt-relevant parts of the two principal agent configurations evaluated in this thesis. The baseline SWE-agent, used in Chapter 4, is configured in `configs/explore_only_v1_artifact_neutral.yaml`. The CLUEFL agent, used in Chapter 5, is configured in `agent/mini-swe-agent/src/minisweagent/config/fault_localization_cluefl_v4.yaml.tmp1`. Both are mini-SWE-agent YAML configurations. Boilerplate Jinja2 formatting helpers common to both configurations (observation truncation and tool-call error messages) are omitted for brevity. The leave-one-component-out variants in Section 5.4.3 derive from the full CLUEFL configuration by removing, in turn, its software-layer knowledge, historical issue retrieval, root-cause taxonomy, or navigation toolbox.

A.1.1. Baseline SWE-agent Prompt

```
system_template: |
  You are an expert repository-level fault localization assistant. Your job
  is to find the repository artifacts most likely to require modification
  to resolve the reported issue.
  Candidate artifacts may include implementation code, tests, documentation,
  build specifications, configuration files, CI workflows, dependency
  manifests, scripts, templates, generated-source definitions, or maintained
  patch files.
  Do not favor source code or any other artifact type by default. Rank
  artifacts only according to issue-specific repository evidence.
  You must NEVER edit existing files or run tests.
  The ONLY file you are allowed to create/write to is: result_N.md
  (where N is the case number from the task)
  You can use: find, grep, sed -n, cat, wc, head, tail, ls, tree, file,
  sort, uniq, awk, xargs.
  NEVER use: go, python, pip, npm, or any build/test/compile commands.
  NEVER use git or inspect commit history, branches, tags, reflogs,
  remotes, diffs from other revisions, or files outside the checked-out
  repository snapshot.
```

```
NEVER read more than 50 lines at a time.

instance_template: |
  Locate and rank at least 10 repository artifacts that are most likely to
  require modification to resolve the issue below.

  ## Issue
  {{task}}

  ## Available Tools
  - `tree -L 2 <folder>/` -- visualize directory structure
  - `find . -type f -not -path './.git/*'` -- enumerate repository
    artifacts without filtering by extension
  - `find . -type f -iname '*keyword*' -not -path './.git/*'` -- find
    paths by a case-insensitive name fragment
  - `grep -rInI --exclude-dir=.git -- 'keyword' .` -- search readable
    artifacts without filtering by extension
  - `grep -rII --exclude-dir=.git -- 'keyword' .` -- list readable
    artifacts containing a keyword
  - `cat <file>` -- read a file (prefer for short files)
  - `sed -n '10,50p' <file>` -- read a specific line range
  - `head -n 50 <file>` / `tail -n 50 <file>` -- read first/last lines
  - `wc -l <file>` -- count lines
  - `ls <folder>/` -- list directory contents
  - Consider all maintained artifact families when relevant, including
    source code, tests, Markdown and other documentation, YAML/JSON/TOML/
    XML configuration, Dockerfiles, Make/CMake files, CI workflows,
    dependency manifests, shell scripts, templates, `.diff` files, and
    `.patch` files.
  - Do not force every artifact family into the ranking; use the issue
    and repository evidence to decide which families are plausible.
  - Treat the current working tree as the complete evidence boundary.
    NEVER use Git history or inspect files outside this repository
    snapshot.
  - NEVER read more than 50 lines at once
  - NEVER create or edit any files (except result_N.md)

  ## Final Output (MANDATORY)
  The task always starts with "[Case N]" where N is a number.
  Before finishing, you MUST write your findings to result_N.md

  # Fault Localization Result

  ## Suspicious Files
  List ALL suspicious artifacts with non-zero confidence. Artifacts not
  listed are assumed score 0.
```

```
| File | Confidence | Reason |
|-----|-----|-----|
| <path/to/most-suspicious-artifact> | 0.85 | <brief reason based on
repository evidence> |
| <path/to/another-artifact> | 0.60 | <brief reason based on repository
evidence> |
| ... (at least 10 artifacts) | ... | ... |
```

Rules for the final file table:

- List ALL artifacts you consider suspicious. Artifacts not listed = score 0.
- Confidence is 0.0-1.0. Sum does NOT need to equal 1.
- Sort by confidence descending.
- Reason must reference specific repository evidence, such as a function or identifier, error string, configuration key, build rule, workflow step, dependency constraint, patch hunk, or documentation inconsistency.

Replace N with the actual case number. Then run:
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT

A.1.2. CLUEFL Prompt

```
agent:
system_template: |
    You are a patient expert fault localization assistant for the large-scale
    open-source Ollama project.
    The only goal is to find the only suspicious file given an issue description
    (source code AND non-code files like Dockerfile, Makefile, configs).

    ## Your External knowledge

    # Knowledge 1: Get project-specific knowledge about Ollama
    python3 __KNOWLEDGELIB__ get_project_specific_knowledge

    # Knowledge 2: Find similar historical issue--fix pairs as few-shot examples
    python3 __KNOWLEDGELIB__ fewshot '<bug description>'

    # Knowledge 3: Get defect type taxonomy for thinking expansion
    python3 __KNOWLEDGELIB__ get_defect_taxonomy

    ## Your Toolbox
```

You have a Python toolbox with 3 commands:

```
```bash
TOOL 1: Get package overview (layer, description, file summaries with func
signatures)
python3 __TOOLBOX__ pkg_summary <package>

TOOL 2: Get Go and C package dependencies (upstream and downstream)
python3 __TOOLBOX__ deps <package>

TOOL 3: Get a specific function's source code (BOTH Go and C/C++ files)
python3 __TOOLBOX__ func <file> <function_name>
```
```

You may also use these shell commands for targeted searches:

```
- `grep -rn 'keyword' --include='*.go' <folder>/` -- search Go files
- `grep -rn 'keyword' --include='*.cpp' <folder>/` -- search C++ files
- `grep -rl 'keyword' <folder>/` -- list only filenames containing a keyword
- `sed -n '10,30p' <file>` -- read a specific line range
- `head -n 50 <file>` / `tail -n 50 <file>` -- read first/last lines
- `find <folder>/ -name '*.go' -type f` -- find files by name pattern
```

Standard Operating Workflow

Phase 1: Hypothesis Generation

1. Read the bug description carefully. Identify symptom, trigger, affected component.
2. Knowledge 1 (get_project_specific_knowledge) -- consult the 8-layer architecture map and critical execution paths to see which layers the symptom could originate from.
3. Knowledge 2 (fewshot) -- find similar historical bugs. Note which files were fixed and why.
4. Knowledge 3 (get_defect_taxonomy) -- consult the 28-category root-cause checklist to consider multiple candidate root causes rather than anchoring on one.
5. Assign every software layer a suspiciousness score in [0,1] based on the map, history, and taxonomy. Layers scoring zero are excluded from further search.

Phase 2: Evidence Exploration

6. Read ALL non-source files in the repository (dependency manifests, build scripts, Dockerfiles, Makefiles, CMake files, configuration files, and other maintained non-source artifacts). This is an exhaustive inspection, not a symptom-dependent reminder.
7. For every software layer with a non-zero suspiciousness score, use Tool 2 (deps) to trace upstream/downstream dependencies and possible cross-layer causal chains.

```
8. Tool 1 (pkg_summary) -- narrow from layers/packages to candidate files using
   file summaries and signatures.
9. Tool 3 (func) -- read source of suspicious functions (Go AND .cpp/.c) to
   confirm or reject hypotheses.
10. grep / sed -- search for error strings, keywords, or patterns not covered by
    the toolbox.

### Phase 3: Relevance Ranking
11. Announce "=== PHASE 3: Final Verdict ==="
12. Assign each candidate file a relevance score in [0,1] with a brief
    justification, then write the final file-level result with python (see output
    template in the task).

## Rules
- The ONLY file you create is: result_00_<n>.md
- Spend at most 3-4 calls on Phase 1. Spend the majority on Phase 2 code
  reading.
- NEVER read more than 50 lines at a time with sed.
- Use `__TOOLBOX__ func` for BOTH Go and C/C++ files.
- Use python to write the result file (not cat/heredoc).

instance_template: |
  ## Issue
  {{task}}

  ## Final Output
  The task always starts with "[Case N]" where N is a number.
  Before finishing, you MUST write your findings to result_N.md.

  Rules for the output:
  - List files with non-zero confidence. Files not listed = score 0.
  - Confidence is 0.0-1.0. Sum does NOT need to equal 1.
  - Sort by confidence descending.
  - Reason must reference specific code evidence (function name, error string,
    logic flaw).

  Then run:
  echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT

  <system_information>
  {{system}} {{release}} {{version}} {{machine}}
  </system_information>
```

Supporting resources. Appendix A.2 summarizes the compact eight-layer operational map supplied to CLUEFL; the accompanying defect-taxonomy resource returns the 28-category taxonomy of Liu et al. [Liu+26b].

A.2. Ollama Software-Layer Taxonomy

Table A.1 summarizes the compact 8-layer architectural map that CLUEFL injects into the agent’s system prompt (Section 5.1.2). This is an Ollama-specific, operational navigation scaffold used by the prompt.

The eight displayed layer counts cover 69 layer-assigned entries. The prompt then summarizes cross-cutting package families, while the separate toolbox dependency catalogue covers 111 package/interface nodes in total (108 Go and three C/C++ interfaces). Consequently, the table below lists representative paths and package families rather than explicitly enumerating all 111 nodes; the “111 packages” label refers to the dependency catalogue’s coverage, not to the sum of the eight displayed rows.

Table A.1.: The compact 8-layer Ollama operational map used by CLUEFL; package paths are representative rather than an exhaustive 111-node list.

| Layer | Name (# entries) | Representative packages/interfaces |
|-------|-----------------------|---|
| 1 | CLI Entry (6) | main.go, cmd/, cmd/config/, cmd/launch/, cmd/tui/ |
| 2 | HTTP API (7) | api/, server/, middleware/, openai/, anthropic/, auth/ |
| 3 | Runner (4) | runner/, runner/llamarunner/, runner/ollamarunner/ |
| 4 | Model (28) | model/, model/models/, model/parsers/, model/renderers/, per-architecture packages (qwen2/3, gemma2/3, deepseek2, llama, ...) |
| 5 | ML Engine (8) | ml/, ml/backend/ggml/, kvcache/, ml/nn/, ml/nn/rope/ |
| 6 | C Bridge (3) | llama/ (CGo bridge), llama/llama.cpp/src/, llama/llama.cpp/common/ |
| 7 | Inference Support (6) | llm/, sample/, template/, thinking/, tokenizer/, tools/ |
| 8 | Infrastructure (7) | discover/, envconfig/, manifest/, logutil/, progress/, version/ |

Cross-cutting concerns that do not belong to a single layer are handled separately: file-system utilities (convert/, fs/), shared helpers (harmony/, parser/, readline/, types/, internal/), and experimental components (x/). Claude Opus 4.6 performs the zero-shot mapping of package and interface nodes to this taxonomy, as described in Section 5.1.2.

Abbreviations

| | |
|---------------|---|
| API | Application Programming Interface |
| APCA | Automated Patch-Correctness Assessment |
| APR | Automated Program Repair |
| AST | Abstract Syntax Tree |
| BM25 | Best Matching 25 |
| CLI | Command-Line Interface |
| CLUEFL | Cross-Software-Layer Understanding of Evidence for Fault Localization |
| CPU | Central Processing Unit |
| CUDA | Compute Unified Device Architecture |
| FL | Fault Localization |
| GPT | Generative Pre-trained Transformer |
| GPU | Graphics Processing Unit |
| GUI | Graphical User Interface |
| HTTP | Hypertext Transfer Protocol |
| IR | Information Retrieval |
| IRBL | Information-Retrieval-Based Bug Localization |
| IRFL | Information-Retrieval-Based Fault Localization |
| LLM | Large Language Model |
| ML | Machine Learning |
| MRR@k | Mean Reciprocal Rank at k |
| PR | Pull Request |
| RQ | Research Question |
| TF-IDF | Term Frequency-Inverse Document Frequency |

List of Figures

| | | |
|------|---|----|
| 4.1. | Construction stages of the Ollama evaluation dataset. | 19 |
| 4.2. | Workflow of the multilingual BugLocator adaptation. | 22 |
| 4.3. | Workflow of the multilingual BLUiR adaptation. | 24 |
| 4.4. | Exclusive Top-1 intersections graph. | 31 |
| 5.1. | Overview of CLUEFL. | 39 |
| 5.2. | Exclusive Top-1 success intersections between SWE-agent and CLUEFL. | 45 |

List of Tables

| | |
|---|----|
| 3.1. Representative benchmarks. Patch-derived labels are observed developer repair locations, not uniquely proved causal locations. | 16 |
| 3.2. Positioning by primary evaluated configuration. | 17 |
| 4.1. Composition of the Ollama evaluation dataset. | 21 |
| 4.2. Overview of the evaluated localization approaches. | 21 |
| 4.3. Principal reproducibility settings. | 25 |
| 4.4. Localization effectiveness on the Ollama evaluation dataset. | 27 |
| 4.5. Average efficiency statistics of the evaluated localization approaches. . . | 28 |
| 4.6. Localization effectiveness by repair-location artifact type. | 32 |
| 5.1. Controlled settings for the RQ4 comparison. | 44 |
| 5.2. CLUEFL Comparison with SWE-agent across different LLMs | 44 |
| 5.3. Matched localization effectiveness by repair-location artifact type across LLM backbones. | 46 |
| 5.4. Leave-one-component-out ablation on the 145 evaluation instances. . . . | 47 |
| A.1. The compact 8-layer Ollama operational map used by CLUEFL; package paths are representative rather than an exhaustive 111-node list. | 60 |

Bibliography

- [Agr+24] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee. “Taming Throughput–Latency Tradeoff in LLM Inference with Sarathi-Serve.” In: *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2024, pp. 117–134.
- [AYM26] M. Asad, R. M. Yasir, and S. Malek. “Towards Explorative IRBL: Combining Semantic Retrieval with LLM-Driven Iterative Code Exploration.” In: *Proceedings of the ACM on Software Engineering* 3.ISSTA (2026). doi: 10.1145/3832150. arXiv: 2508.00253 [cs.SE].
- [CCC26] G. Caumartin, T.-H. P. Chen, and D. E. Costa. *Retrieval-Oriented Code Representations in Agentic Bug Localization*. 2026. arXiv: 2607.11046 [cs.SE].
- [Cha+25] J. Chang, X. Zhou, L. Wang, D. Lo, and B. Li. *Bridging Bug Localization and Issue Fixing: A Hierarchical Localization Framework Leveraging Large Language Models*. BugCerberus. 2025. arXiv: 2502.15292 [cs.SE].
- [Che+25] Z. Chen, R. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang. “LocAgent: Graph-Guided LLM Agents for Code Localization.” In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 8697–8727. doi: 10.18653/v1/2025.acl-long.426.
- [CM26] G. Caumartin and G. Melo. “Reformulate, Retrieve, Localize: Agents for Repository-Level Bug Localization.” In: *Proceedings of the 7th International Workshop on Bots and Agents in Software Engineering*. 2026, pp. 39–45. doi: 10.1145/3786161.3788460.
- [Fen+26] Q. Feng, X. Ma, J. Sheng, Z. Feng, W. Song, and P. Liang. “Integrating Various Software Artifacts for Better LLM-Based Bug Localization and Program Repair.” In: *ACM Transactions on Software Engineering and Methodology* 35.7 (2026), pp. 1–25. doi: 10.1145/3770581. arXiv: 2412.03905 [cs.SE].
- [FG26] M. Farzandway and F. Ghassemi. *SieveFL: Hierarchical Runtime-Aware Pruning for Scalable LLM-Based Fault Localization*. 2026. arXiv: 2605.13491 [cs.SE].

- [Fus+26] M. Fuster-Pena, D. de-Fitero-Dominguez, A. Garcia-Cabot, and E. Garcia-Lopez. *RePaCA: Leveraging Reasoning Large Language Models for Static Automated Patch Correctness Assessment*. Preprint version available at the review cutoff; forthcoming in *Neurocomputing*, vol. 701, article 134583. 2026. doi: 10.1016/j.neucom.2026.134583. arXiv: 2507.22580 [cs.SE].
- [Gu+26] Y. Gu, Z. Zhang, S. Zhu, X. Fu, J. Wu, Y. Wang, and B. Kasikci. *Ekka: Automated Diagnosis of Silent Errors in LLM Inference*. 2026. doi: 10.48550/arXiv.2606.04594. arXiv: 2606.04594 [cs.DC].
- [Jia+25] Z. Jiang, X. Ren, M. Yan, W. Jiang, Y. Li, and Z. Liu. "Issue Localization via LLM-Driven Iterative Code Graph Searching." In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 2025, pp. 3034–3045. doi: 10.1109/ASE63991.2025.00249. arXiv: 2503.22424 [cs.SE].
- [Jim+24] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" In: *International Conference on Learning Representations*. 2024. arXiv: 2310.06770.
- [JJE14] R. Just, D. Jalali, and M. D. Ernst. "Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs." In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. ACM, 2014, pp. 437–440. doi: 10.1145/2610384.2628055.
- [Kan+25] S. Kang, S. Yun, J. Hong, S. Yoo, and G. An. *Finding the Needle in the Crash Stack: Industrial-Scale Crash Root Cause Localization with AutoCrashFL*. Preprint under review at the literature cutoff. 2025. arXiv: 2510.22530 [cs.SE].
- [KAY24] S. Kang, G. An, and S. Yoo. "A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization." In: *Proceedings of the ACM on Software Engineering* 1.FSE (2024), pp. 1424–1446. doi: 10.1145/3660771.
- [Kim+07] S. Kim, T. Zimmermann, E. J. Whitehead, and A. Zeller. "Predicting Faults from Cached History." In: *Proceedings of the 29th International Conference on Software Engineering*. ICSE '07. IEEE Computer Society, 2007, pp. 489–498. doi: 10.1109/ICSE.2007.66.
- [Kim+25] D. Kim, M. Jeon, D. Hwang, and H. Oh. "PAFL: Enhancing Fault Localizers by Leveraging Project-Specific Fault Patterns." In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA1 (2025). doi: 10.1145/3720526.

- [Ko+06] A. J. Ko, B. A. Myers, M. J. Coblenz, and H. H. Aung. "An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks." In: *IEEE Transactions on Software Engineering* 32.12 (2006), pp. 971–987. doi: 10.1109/TSE.2006.116.
- [Koc+16] P. S. Kochhar, X. Xia, D. Lo, and S. Li. "Practitioners' Expectations on Automated Fault Localization." In: *Proceedings of the 25th International Symposium on Software Testing and Analysis*. ISSTA 2016. ACM, 2016, pp. 165–176. doi: 10.1145/2931037.2931051.
- [Kwo+23] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. "Efficient Memory Management for Large Language Model Serving with PagedAttention." In: *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.
- [Law+13] J. Lawrance, C. Bogart, M. Burnett, R. K. E. Bellamy, K. Rector, and S. D. Fleming. "How Programmers Debug, Revisited: An Information Foraging Theory Perspective." In: *IEEE Transactions on Software Engineering* 39.2 (2013), pp. 197–215. doi: 10.1109/TSE.2010.111.
- [LC26] H. Li and M. Coblenz. "A Grounded Theory of Debugging in Professional Software Engineering Practice." In: *Proceedings of the ACM on Software Engineering* 3.FSE (2026), pp. 1082–1103. doi: 10.1145/3797077.
- [Lee+18] J. Lee, D. Kim, T. F. Bissyandé, W. Jung, and Y. Le Traon. "Bench4BL: Reproducibility Study on the Performance of IR-Based Bug Localization." In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2018. ACM, 2018, pp. 61–72. doi: 10.1145/3213846.3213856.
- [Li+25] Z. Li, Z. Jiang, Q. Huang, and Q. Gu. "LLM-BL: Large Language Models Are Zero-Shot Rankers for Bug Localization." In: *2025 IEEE/ACM 33rd International Conference on Program Comprehension*. IEEE, 2025, pp. 548–559. doi: 10.1109/ICPC66645.2025.00064.
- [Liu+26a] J. Liu, Y. Lin, C. Liu, Y. Qian, Y. Liu, J. Chang, W. Zhang, and L. Huang. "IssueExec: A Test-Driven Approach for Localizing Software Engineering Issues." In: *Proceedings of the ACM on Software Engineering* 3 (2026). ISSTA 2026. doi: 10.1145/3832290.
- [Liu+26b] M. Liu, S. Zhong, W. Bi, Y. Zhang, Z. Chen, Z. Chen, X. Liu, and Y. Ma. "A First Look at Bugs in LLM Inference Engines." In: *ACM Transactions on Software Engineering and Methodology* (2026). doi: 10.1145/3788873. arXiv: 2506.09713 [cs.SE].

- [Liu+26c] W. Liu, C. Peng, P. Gao, A. Liu, W. Zhang, H. Zhao, and Z. Jin. “GraphLocator: Graph-Guided Causal Reasoning for Issue Localization.” In: *Proceedings of the ACM on Software Engineering* 3.FSE (2026), pp. 1128–1149. DOI: 10.1145/3797079. arXiv: 2512.22469 [cs.SE].
- [LM10] T. D. LaToza and B. A. Myers. “Developers Ask Reachability Questions.” In: *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*. ICSE ’10. Association for Computing Machinery, 2010, pp. 185–194. DOI: 10.1145/1806799.1806829.
- [LW26] H. Li and Y. Wang. “Reliability of LLM Inference Engines from a Static Perspective: Root Cause Analysis and Repair Suggestion via Natural Language Reports.” In: *Big Data and Cognitive Computing* 10.2 (2026). DOI: 10.3390/bdcc10020060.
- [Ma+26] D. Ma, S. Chen, Y. Yang, Y. Shi, Y. Yan, and X. Gu. *LLM Agents Can See Code Repositories*. SeeRepo. 2026. arXiv: 2606.14061 [cs.SE].
- [MU26] M. A. A. Mamun and G. Uddin. “BLAgent: Agentic RAG for File-Level Bug Localization.” In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3830238. arXiv: 2605.17965 [cs.SE].
- [Niu+25] F. Niu, C. Li, K. Liu, X. Xia, and D. Lo. “When Deep Learning Meets Information Retrieval-Based Bug Localization: A Survey.” In: *ACM Computing Surveys* 57.11 (2025), pp. 1–41. DOI: 10.1145/3734217.
- [NY25] G. Nam and G. Yang. “LLMLoc: A Structure-Aware Retrieval System for Zero-Shot Bug Localization.” In: *Electronics* 14.21 (2025), p. 4343. DOI: 10.3390/electronics14214343.
- [Oll26a] Ollama. *Ollama*. GitHub repository. 2026.
- [Oll26b] Ollama. *Ollama: All Aboard Open Models*. Ollama Blog. Ollama reports serving 8.9 million developers. July 2026.
- [OWB05] T. J. Ostrand, E. J. Weyuker, and R. M. Bell. “Predicting the Location and Number of Faults in Large Software Systems.” In: *IEEE Transactions on Software Engineering* 31.4 (2005), pp. 340–355. DOI: 10.1109/TSE.2005.49.
- [Por80] M. F. Porter. “An Algorithm for Suffix Stripping.” In: *Program* 14.3 (1980), pp. 130–137. DOI: 10.1108/eb046814.
- [PT15] M. Papadakis and Y. L. Traon. “Metallaxis-FL: Mutation-Based Fault Localization.” In: *Software Testing, Verification and Reliability* 25.5–7 (2015), pp. 605–628. DOI: 10.1002/stvr.1509.

- [Qin+24] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao. *AgentFL: Scaling LLM-Based Fault Localization to Project-Level Context*. 2024. DOI: 10.48550/arXiv.2403.16362. arXiv: 2403.16362 [cs.SE].
- [Qin+25a] Y. Qin, S. Wang, Y. Lei, Z. Zhang, B. Lin, X. Peng, L. Chen, and X. Mao. “Fault Localization from the Semantic Code Search Perspective.” In: *ACM Transactions on Software Engineering and Methodology* (2025). CosFL; author manuscript arXiv:2411.17230. DOI: 10.1145/3757915.
- [Qin+25b] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao. “SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization.” In: *IEEE Transactions on Software Engineering* 51.4 (2025), pp. 1173–1187. DOI: 10.1109/TSE.2025.3543187.
- [Red+26] R. G. Reddy, T. Suresh, J. Doo, Y. Liu, X.-P. Nguyen, Y. Zhou, S. Yavuz, C. Xiong, H. Ji, and S. Joty. “SweRank: Software Issue Localization with Code Ranking.” In: *The Fourteenth International Conference on Learning Representations*. 2026. arXiv: 2505.07849 [cs.SE].
- [RZ09] S. Robertson and H. Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond.” In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389. DOI: 10.1561/15000000019.
- [Sah+13] R. K. Saha, M. Lease, S. Khurshid, and D. E. Perry. “Improving Bug Localization Using Structured Information Retrieval.” In: *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2013, pp. 345–355. DOI: 10.1109/ASE.2013.6693093.
- [Sch+26] D. Scheinert, A. Acker, T. Wittkopp, S. Becker, H. Yous, K. Reddy, I. Farhat, H. Hacid, and O. Kao. *Beyond Microservices: Testing Web-Scale RCA Methods on GPU-Driven LLM Workloads*. 2026. DOI: 10.48550/arXiv.2603.02057. arXiv: 2603.02057 [cs.DC].
- [Sed+26] S. Seddik, F. Seddik, A. Esmaili, M. Sadatbenis, and F. Fard. *ARISE: A Repository-Level Graph Representation and Toolset for Agentic Program Repair and Fault Localization*. Version 2. 2026. DOI: 10.48550/arXiv.2605.03117. arXiv: 2605.03117 [cs.SE].
- [Sep+26] M. Sepidband, H. Taherkhani, H. V. Pham, and H. Hemmati. *RGFL: Reasoning Guided Fault Localization for Automated Program Repair Using Large Language Models*. 2026. DOI: 10.48550/arXiv.2601.18044. arXiv: 2601.18044 [cs.SE].

- [Sha+26a] S. Shao, D. Wang, Y. Zeng, and T. Yu. “ConFL: Explainable Concurrent Fault Localization via Hierarchy-Guided LLM Reasoning.” In: *Proceedings of the ACM on Software Engineering* 3 (2026). ISSTA 2026; arXiv:2608.02974. DOI: 10.1145/3832168.
- [Sha+26b] S. Shao, Y. Zeng, Y. Zhao, and T. Yu. *HyperFL: Query-Adaptive Representation Learning for Software Fault Localization*. 2026. arXiv: 2608.02967 [cs.SE].
- [SLC25] X. Shi, Z. Li, and A. R. Chen. *Enhancing LLM-Based Fault Localization with a Functionality-Aware Retrieval-Augmented Generation Framework*. FaR-Loc. 2025. arXiv: 2509.20552 [cs.SE].
- [SPH26] M. Sepidband, H. V. Pham, and H. Hemmati. *On the Role of Fault Localization Context for LLM-Based Program Repair*. 2026. DOI: 10.48550/arXiv.2604.05481. arXiv: 2604.05481 [cs.SE].
- [SR25] A. M. Samir and M. M. Rahman. *Improved IR-Based Bug Localization with Intelligent Relevance Feedback*. 2025. DOI: 10.48550/arXiv.2501.10542. arXiv: 2501.10542 [cs.SE].
- [SWE26] SWE-agent Team. *mini-SWE-agent: The Minimal AI Software Engineering Agent*. GitHub repository. 2026.
- [SY24] S. Shao and T. Yu. *Enhancing IR-Based Fault Localization Using Large Language Models*. LLmiRQ and LLmiRQ+. 2024. DOI: 10.48550/arXiv.2412.03754. arXiv: 2412.03754 [cs.SE].
- [Tan+26] G. Tan, Z. He, M. Li, P. Chen, J. Shi, Z. Sun, T. Zhang, D. Chen, L. K. Shar, C. Zhang, and D. Lo. *LIDL: LLM Integration Defect Localization via Knowledge Graph-Enhanced Multi-Agent Analysis*. 2026. DOI: 10.48550/arXiv.2601.05539. arXiv: 2601.05539 [cs.SE].
- [THA23] S. Tsumita, S. Hayashi, and S. Amasaki. “Large-Scale Evaluation of Method-Level Bug Localization with FinerBench4BL.” In: *Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2023, pp. 815–824. DOI: 10.1109/SANER56733.2023.00094.
- [Wan+24] X. Wang, B. Li, Y. Song, F. F. Xu, et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. 2024. arXiv: 2407.16741 [cs.SE].
- [Wan+26] H. Wang, R. Yu, Y. Pang, Y. Zhao, J. Chen, J. Jiang, and Z. Wang. “Towards Better Linux Kernel Fault Localization: Leveraging Contrastive Reasoning and Hierarchical Context Analysis.” In: *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. ICSE 2026; arXiv:2607.00562. 2026. DOI: 10.1145/3744916.3787792.

- [WL14] S. Wang and D. Lo. “Version History, Similar Report, and Structure: Putting Them Together for Improved Bug Localization.” In: *Proceedings of the 22nd International Conference on Program Comprehension*. ICPC 2014. Association for Computing Machinery, 2014, pp. 53–63. doi: 10.1145/2597008.2597148.
- [Won+16] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa. “A Survey on Software Fault Localization.” In: *IEEE Transactions on Software Engineering* 42.8 (2016), pp. 707–740. doi: 10.1109/TSE.2016.2521368.
- [Wu+23] Y. Wu, Z. Li, J. M. Zhang, M. Papadakis, M. Harman, and Y. Liu. *Large Language Models in Fault Localisation*. 2023. doi: 10.48550/arXiv.2308.15276. arXiv: 2308.15276 [cs.SE].
- [Xi+26] H. Xi, M. Shao, B. Dolan-Gavitt, M. Shafique, and R. Karri. *From Trace to Line: LLM Agent for Real-World OSS Vulnerability Localization*. Version 3; T2L-Agent. 2026. arXiv: 2510.02389 [cs.SE].
- [Xia+16] X. Xia, L. Bao, D. Lo, and S. Li. ““Automated Debugging Considered Harmful” Considered Harmful: A User Study Revisiting the Usefulness of Spectra-Based Fault Localization Techniques with Professionals Using Real Bugs from Large Systems.” In: *Proceedings of the 2016 IEEE International Conference on Software Maintenance and Evolution*. ICSME 2016. IEEE, 2016, pp. 267–278. doi: 10.1109/ICSME.2016.67.
- [Xia+24] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. *Agentless: Demystifying LLM-Based Software Engineering Agents*. 2024. arXiv: 2407.01489 [cs.SE].
- [Xu+25] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo. “FlexFL: Flexible and Effective Fault Localization with Open-Source Large Language Models.” In: *IEEE Transactions on Software Engineering* 51.5 (2025), pp. 1455–1471. doi: 10.1109/TSE.2025.3553363.
- [Yan+21] Z. Yang, J. Shi, S. Wang, and D. Lo. “IncBL: Incremental Bug Localization.” In: *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*. 2021. doi: 10.1109/ASE51524.2021.9678546.
- [Yan+24a] A. Z. H. Yang, C. L. Goues, R. Martins, and V. J. Hellendoorn. “Large Language Models for Test-Free Fault Localization.” In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. ACM, 2024, pp. 1–12. doi: 10.1145/3597503.3623342.

- [Yan+24b] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press. “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.” In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 50528–50652. arXiv: 2405.15793 [cs.SE].
- [Yan+26] Z. Yang, H. Zhu, Q. Zhang, R. Gupta, and A. Kundu. *SemLoc: Structured Grounding of Free-Form LLM Reasoning for Fault Localization*. 2026. DOI: 10.48550/arXiv.2603.29109. arXiv: 2603.29109 [cs.SE].
- [YPD26] H. Yang, Y. Peng, and Z. Ding. “EffiHolmes: Differential Profiling-Guided Repository Level Time Inefficiency Fix Localization.” In: *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ASE 2026 research track; arXiv:2608.03558. 2026. DOI: 10.1145/3832783.3834353.
- [YRB25] I. Yeo, D. Ryu, and J. Baik. *Improving LLM-Based Fault Localization with External Memory and Project Context*. 2025. DOI: 10.48550/arXiv.2506.03585. arXiv: 2506.03585 [cs.SE].
- [Yu+22] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. “Orca: A Distributed Serving System for Transformer-Based Generative Models.” In: *16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2022, pp. 521–538.
- [Yu+25] Z. Yu, H. Zhang, Y. Zhao, H. Huang, M. Yao, K. Ding, and J. Zhao. *OrcaLoca: An LLM Agent Framework for Software Issue Localization*. 2025. arXiv: 2502.00350 [cs.SE].
- [Zha+24] Q. Zhang, C. Fang, W. Sun, Y. Liu, T. He, X. Hao, and Z. Chen. “APPT: Boosting Automated Patch Correctness Prediction via Fine-Tuning Pre-Trained Models.” In: *IEEE Transactions on Software Engineering* 50.3 (2024). arXiv:2301.12453, pp. 474–494. DOI: 10.1109/TSE.2024.3354969.
- [Zha+25] J. Zhang, J. Lian, H. Qin, M. Zhou, K. Lu, R. Mao, and H. Liao. “Hierarchical Reward Modeling for Fault Localization in Large Code Repositories.” In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 17782–17796. DOI: 10.18653/v1/2025.findings-emnlp.966.
- [Zha+26a] S. Zhan, S. Hu, B. Feng, H. Lin, A. Gong, Z. Zhou, J. Zhou, Y. Hou, H. Su, and H.-T. Zheng. *MM-IssueLoc: A Controlled Benchmark for Evaluating Visual Evidence in Multimodal Repository-Level Issue Localization*. 2026. DOI: 10.48550/arXiv.2607.15205. arXiv: 2607.15205 [cs.SE].

- [Zha+26b] Z. Zhang, Y. Duan, Y. Zhang, Y. Xu, Z. Wang, K. Liang, W. Li, J. Liang, D. Xia, J. Huang, J. He, and Y. Wu. *One Tool Is Enough: Reinforcement Learning for Repository-Level LLM Agents*. Version 6. 2026. arXiv: 2512.20957 [cs.SE].
- [Zha+26c] Y. Zhao, Y. Zhao, Y. Zhang, Z. Liu, and M. L. Mazurek. *Continuous Discovery of Vulnerabilities in LLM Serving Systems with Fuzzing*. 2026. DOI: 10.48550/arXiv.2605.11202. arXiv: 2605.11202 [cs.CR].
- [Zhe+24] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. “SGLang: Efficient Execution of Structured Language Model Programs.” In: *Advances in Neural Information Processing Systems*. Vol. 37. Curran Associates, Inc., 2024, pp. 62557–62583. DOI: 10.52202/079017-2000. arXiv: 2312.07104 [cs.DC].
- [Zho+26] Z. Zhou, Z. Huang, Y. He, C. Wang, J. Wang, Y. Wu, X. Peng, and Y. Lou. “Taming System Complexity: Demystifying Software Engineering Agents in Diagnosing Linux Kernel Faults.” In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, 2026, pp. 18899–18916. DOI: 10.18653/v1/2026.acl-long.862. arXiv: 2505.19489 [cs.SE].
- [ZZL12] J. Zhou, H. Zhang, and D. Lo. “Where Should the Bugs Be Fixed? More Accurate Information Retrieval-Based Bug Localization Based on Bug Reports.” In: *Proceedings of the 34th International Conference on Software Engineering*. ICSE ’12. IEEE, 2012, pp. 14–24. DOI: 10.1109/ICSE.2012.6227210.